7 天用 Xcode 学会 C++：完整学习方案

专门为"想用 Xcode 在 Mac 上从零学会 C++"的你设计。

每天 4～6 小时，循序渐进、环环相扣，从打开 Xcode 到独立完成完整项目。

本套资料包含：**每日手册 + 可运行代码 + 分级练习 + 自测 + 打卡清单 + 桌面学习软件。**

这套资料怎么用

```
C++七天学习计划/
├── README.md                    ← 你正在看的：总览、Xcode 配置、使用说明（先读这个）
├── 第 1 天_Xcode 入门与 C++基础.md
├── 第 2 天_流程控制与循环.md
├── 第 3 天_函数与数组字符串.md
├── 第 4 天_指针引用与动态内存.md
├── 第 5 天_类与对象.md
├── 第 6 天_继承多态与 STL.md
├── 第 7 天_综合项目与查漏补缺.md
├── 完整学习方案.pdf             ← 全部内容合并成的一份精美 PDF
├── 完整学习方案.docx            ← Word 版，方便自己做笔记
└── code/                        ← 每天一份可直接运行的源码
    ├── day01_hello.cpp  ~  day07_final_project.cpp
```

三种学习方式，任选一种：

方式	适合谁	入口
桌面学习软件	最喜欢"点一点就能学"	桌面上的「C++七天学习」
阅读手册 + Xcode 练习	想边读边动手	本文件夹的 7 份手册 + code/
打印 PDF	想离线精读、做笔记	完整学习方案.pdf

7 天总览（每天都是"回顾 → 新知识 → 动手 → 练习 → 自测 → 打卡"）

天	主题	核心内容	当天产出	预计时间

第 1 天	Xcode 入门 + C++ 基础	建项目、程序结构、变量、5 种类型、输入输出、运算符、断点调试	温度转换器	4～5 小时
第 2 天	流程控制	if/switch、比较/逻辑运算符、for/while/do-while、break/continue	猜数字游戏	4～5 小时
第 3 天	函数、数组、字符串	函数、值传递、默认参数、重载、递归、数组、冒泡排序、string	排序 + 二分查找	5～6 小时
第 4 天	指针、引用、动态内存	地址、指针、解引用、野指针、引用、new/delete、内存泄漏	指针练习 + 动态数组	5～6 小时
第 5 天	类与对象	class、封装、构造/析构、this、静态成员、对象数组	银行账户系统	5～6 小时
第 6 天	继承、多态 + STL	继承、虚函数、纯虚函数、运算符重载、vector、sort、map	员工工资体系	5～6 小时
第 7 天	综合项目 + 查漏补缺	文件读写 + 学生成绩管理系统 + 7 天自查 + 进阶路线	完整管理系统	6～8 小时

设计原则：每一天都在"回顾昨天"的基础上盖新楼；每个新概念都先讲"为什么需要它"，再讲"是什么"，再给"马上动手"的小任务，最后用分级练习巩固。**节奏平缓，不跳跃。**

每天的标准学习流程（约 4～6 小时）

步骤	做什么	时间
1	读当天的「路线图」，心里有数今天要学什么	5 分钟
2	跟着手册的分步讲解，理解每一个概念（含"为什么"）	60～90 分钟
3	在 Xcode 里运行当天的	60～90 分钟

	code/day0X_*.cpp，**亲手敲一遍**	
4	完成当天「马上动手」小任务（手册里嵌着答案）	30 分钟
5	做「分级练习」（基础/进阶/挑战）	40～60 分钟
6	做「自测」+ 过一遍「坑点汇总」	10 分钟
7	完成「打卡清单」，写一句今日总结	5 分钟

> **黄金法则：示例代码一定要亲手敲，不要只复制粘贴。敲一遍 = 看十遍。**

Xcode 快速配置（1 分钟版）

详细图文版在第 1 天手册的"第 1 步"里，这里给速查：

1. 打开 Xcode → **File** → **New** → **Project...**（Cmd + Shift + N）
2. 先点上方 **macOS** 标签 → 选 **Command Line Tool** → Next
3. **Product Name** 填 CppLearning，**Language 必须选 C++**（默认是 Swift！）
4. 选择保存位置 → Create
5. 把 code/day0X_*.cpp 的内容复制进 main.cpp
6. 按 **`Cmd + R`** 运行，结果在底部调试区；要输入时先点击调试区

调试三件套（每天都要用）：

- **打断点**：点击行号左侧，出现蓝色箭头
- **单步调试**：Continue（继续）/ Step Over（跳过）/ Step Into（进入函数）/ Step Out（跳出）
- **看变量**：调试区左侧 Variables 面板实时显示每个变量的值

桌面学习软件使用说明

桌面上的「C++七天学习」把全部内容做成了交互式软件：

板块	**功能**
首页总览	7 天课程总表、总进度、一键"继续学习"
课程页	完整手册渲染 + 可勾选的打卡清单 + 进度环
代码实验室	内置 7 份示例代码，**改代码** → **编译** → **运行**，还能给程序输入内容
进度系统	打卡自动累计，重启不丢失

软件里的内容和本文件夹完全一致。想改软件本身，用 Xcode 打开

学习软件源码/CppLearningApp.xcodeproj 即可（源码注释齐全）。

常见问题 FAQ

Q1：Xcode 里 Language 只有 Swift，找不到 C++?

新建项目时先点上方 "macOS" 标签，选 "Command Line Tool"，Next 之后才会出现 Language 下拉框，选 C++。

Q2：代码粘贴进去报一堆红错?

多半是建成了 Swift 项目。重新建项目，Language 选 C++。

Q3：程序运行了但看不到输出?

按 Cmd + Shift + Y 打开底部调试区。

Q4：程序要求输入，但打字没反应?

点击底部调试区，让光标聚焦在控制台上再输入。

Q5：cin 输入后，后面的输入"跳过去"了?

cin >> 会在缓冲区留下换行符，后面用 getline 前要加 cin.ignore()（第 3 天有详解）。

Q6：第 4 天指针太难，看不懂怎么办?

正常！指针是公认的难点。把"信箱/门牌号"的比喻抄下来、画图理解，再打断点一步步看。第 4 天可以拆成两个晚上。

Q7：每天时间不够怎么办?

学编程最忌贪多。每天只求"学透 + 完成练习"，7 天变 10 天也没关系，基础牢最重要。

Q8：这个学习过程需要联网吗?

不需要。手册、代码、软件全部在本地，编译和运行都用你电脑上的 Xcode。只有后续进阶（LeetCode、查文档）才需要网络。

学习心法（读 3 遍）

1. **动手 > 观看**：所有示例亲手敲，练习先自己想，再看答案
2. **会调试 > 会背语法**：报错是老师，断点是放大镜
3. **理解"为什么" > 死记"怎么做"**：每学一个概念，先问它解决什么问题
4. **输出是最好的学习**：每天结束，用一句话向自己复述今天学会了什么
5. **接受反复**：一个概念今天不懂，睡一觉、隔天再看，往往就通了

7 天后的验收标准

学完这一周，你应该能：

- 不查资料写出"猜数字"游戏
- 讲清楚指针、值、地址的关系
- 用类封装现实概念（学生、账户、圆）
- 用 vector + sort 处理数据
- 独立演示"学生成绩管理系统"（增删改查 + 排序 + 文件保存）
- 读懂编译报错，用断点定位问题

如果这 6 条都做到了，恭喜——你已经入门 C++，具备了继续深入的一切基础！

祝你学习顺利！任何一天卡住了，把报错信息或代码截图发给我，我们一起解决。

预备课：零基础友好入门

预计用时：30～60 分钟 | 这一课不写代码，只回答三个问题：

编程是什么？学 C++ 要准备什么？怎么学才能学会？

哪怕你完全没接触过编程，上完这课也会心里有底。

上完这课，你能做到（打卡清单）

- [ ] 用自己的话解释"程序"和"编程"是什么
- [ ] 明白"编译"和"运行"是怎么回事
- [ ] 知道学 C++ 需要准备什么（其实只需要电脑 + Xcode）
- [ ] 破除三个常见恐惧：数学不好、英语不好、怕记不住
- [ ] 知道正确的学习方法：动手 > 观看

一、编程到底是在做什么？

1.1 打个比方：编程 = 给电脑写"说明书"

电脑本身很"笨"，它只会一步一步执行指令，而且必须说得**非常具体、非常清楚**。

比如你想让电脑"把 1 加到 100"，你得告诉它：

1. 准备一个盒子，先放 0
2. 拿 1 放进盒子
3. 拿 2 加进去
4. 拿 3 加进去
5. ……一直加到 100

编程就是写这样一份"说明书"，只不过用的是电脑听得懂的"语言"（比如 C++）。

大白话：**程序 = 给电脑的一步步指令；编程 = 写这些指令。**

1.2 为什么有这么多"编程语言"？

因为人和电脑沟通有不同的"方言"：C++、Python、Java、Swift……

- 有的语言像"高数"（离电脑近、功能强、难一点）→ C++ 属于这种
- 有的语言像"大白话"（简单、写起来快）→ Python 属于这种

为什么推荐先学 C++？ 因为它让你真正理解电脑是怎么工作的（内存、地址、指针），学会它之后再学别的语言会非常轻松。就像先学手动挡再开自动挡，技术更扎实。

1.3 "编译"和"运行"是什么？

电脑只认识 0 和 1 组成的机器语言，不认识我们写的英文/符号代码。所以：

```
你写的 C++ 代码（人看的）
   ↓ 编译（翻译成机器语言，Xcode 帮你做）
机器语言程序（电脑看的）
   ↓ 运行（电脑执行）
运行结果（屏幕上的输出）
```

- **编译**：把代码"翻译"成电脑能执行的程序。有语法错误时，这一步会报错
- **运行**：让翻译好的程序真正跑起来

在 Xcode 里按 Cmd + R，就是"先编译、再运行"一步搞定。报错是电脑在说"这句我没看懂"，改对就好了——**报错不是失败，是提示。**

二、学 C++ 需要准备什么？

只需要三样：

需要	说明	你有吗
一台 Mac	运行 Xcode 用	肯定有（你正在用）
Xcode	免费的开发工具（App Store 可下载）	已经装好了
一份耐心	每天 1～2 小时，坚持 7 天	从现在开始培养

不需要买课、不需要额外软件、不需要联网（本地就能学）。

三、破除三个常见恐惧

恐惧 1："我数学不好，能学编程吗？"

能。 编程主要用的是"逻辑"，不是高数。这 7 天用到的数学只有：加减乘除、比较大小、取余数——小学水平就够了。真正的难点在"把大问题拆成小步骤"，这和数学成绩无关。

恐惧 2："我英语不好，能学吗？"

能。 常用的英文单词就几十个（if、for、int、cout……），而且：

- 大多数字母就 2～6 个，比记单词简单多了
- 教程里每个英文都会用中文解释
- 实在记不住就查"术语速查"（软件里有）

恐惧 3："我怕记不住、怕学不会。"

这是所有人的感受，包括程序员。 三个事实帮你放下包袱：

1. **没人靠"背"学会编程**，都是靠"用"
2. **报错、忘记、卡住，都是学习过程的一部分**，不是"你不行"
3. 学编程就像学骑车：看再多视频不会骑，**上去骑、摔几次，就会了。**教程里的每一段代码，都值得你亲手敲一遍

四、正确的学习方法（很重要！）

一句话：动手 > 观看。

错误做法	正确做法
只看教程，觉得"看懂了"	每段示例代码亲手敲一遍
一上来背语法	先做小练习，用到什么查什么
卡住就放弃	卡住先打断点、看报错，再问
一天学完 7 天内容	每天只学一章，练熟再前进

每天 30 分钟 > 每周 5 小时突击。 细水长流，7 天后你会惊讶自己的进步。

五、怎么用这个软件学习

1. 从左侧「第 1 天」开始，按顺序学

2. 每章都有：**知识点讲解 → 动画演示 → 马上动手 → 练习 → 自测**
3. 遇到看不懂的术语，随时点左侧「C++ 术语速查」查一下
4. 打开「代码实验室」可以直接改代码、点"编译并运行"看效果
5. 每天学完，勾选页面的「打卡清单」，进度会保存

建议节奏：

时间	做什么
前 30 分钟	读当天的讲解 + 看动画
中间 60 分钟	在代码实验室亲手敲代码、跑程序
后 30 分钟	做练习和自测，勾打卡
睡前 5 分钟	回忆今天学了什么（说给自己听）

预备课小结

- 编程 = 给电脑写"说明书"
- C++ = 离电脑近、难一点但很扎实的语言
- 编译 = 翻译，运行 = 执行
- 你不需要数学好、英语好，只需要**动手 + 坚持**
- 这套软件已经帮你把路铺好了，从第 1 天开始走就好

现在去点开「第 1 天」吧！第一步先学会在 Xcode 里运行你的第一个程序

第 1 天：Xcode 入门 + C++ 基础语法

预计用时：4～5 小时 ｜ 今天是第一天，也是最重要的一天：把"环境 + 基本语法 + 调试"一次打通。

学完之后，你就能在 Xcode 里独立写出"输入 → 处理 → 输出"的小程序。

今天学完，你能做到（打卡清单）

- [ ] 在 Xcode 中新建 C++ 命令行项目并运行 Hello World
- [ ] 逐行看懂 C++ 程序的基本结构（#include、main、return 0）
- [ ] 会用 cout 输出文字和变量
- [ ] 认识并会使用 5 种基本类型：int、double、char、bool、string
- [ ] 会用 cin 从键盘读取输入
- [ ] 会用加减乘除和常见运算符，避开整数除法的大坑
- [ ] 会打断点、单步调试、查看变量值

今天的路线图（跟着走就不会乱）

今天的知识是一层层盖起来的，每一步都在为下一步打地基：

```
第 1 步 配置 Xcode（工具就绪）
  ↓
第 2 步 认识程序结构（看懂 Hello World）
  ↓
第 3 步 cout 输出（让程序说话）
  ↓
第 4 步 变量与类型（让程序记住东西）
  ↓
第 5 步 cin 输入（让程序听懂你）
  ↓
第 6 步 运算符（让程序会算数）
  ↓
第 7 步 综合小项目：温度转换器（把前 6 步串起来）
  ↓
```

第 8 步 断点调试（学会自己找 bug）

建议每完成一步，就停下来**亲手敲一遍示例**再继续，不要一口气看完。

第 1 步：把 Xcode 配置好（约 20 分钟）

第 1 小步：打开 Xcode

在"启动台"里找到 Xcode 图标并打开。第一次启动时，系统会提示安装额外组件，点击"同意"并等待完成（几分钟）。这一步只需要做一次。

第 2 小步：新建一个 C++ 项目

1. 菜单栏选 **File** → **New** → **Project...**（快捷键 Cmd + Shift + N）
2. 在弹出的窗口里，**先点最上方的 "macOS" 标签**（不是 iOS！）
3. 在模板列表里找到 **Command Line Tool**（命令行工具），选中后点 **Next**
4. 填写项目信息：

- **Product Name（产品名称）**：填 CppLearning
- **Language（语言）**：这里最关键！下拉框默认是 Swift，**必须改成 C++**
- 其余项保持默认即可

5. 点 **Next**，选择保存位置（建议放"文稿"文件夹里建一个 CppProjects 文件夹统一管理），点 **Create**

> 最常犯的错：Language 忘选 C++，建成了 Swift 项目。如果发现代码颜色、内容不对，八成是这个原因，删掉重建即可。

第 3 小步：认识 Xcode 的 4 个区域

- **左侧导航区**：显示项目里的文件，我们主要用 main.cpp
- **中间编辑区**：写代码的地方（光标在这里）
- **底部调试区**：运行结果显示在这里，快捷键 Cmd + Shift + Y 可以显示/隐藏
- **顶部工具栏**：右侧的 ▶️ 是运行按钮

第 4 小步：运行你的第一个程序

新建好的 main.cpp 里已经有一段代码。直接按 **`Cmd + R`**（或点 ▶️），几秒后底部调试区会出现一行字：

```
Hello, World!
```

恭喜！你的 C++ 开发环境已经正式跑通了

> **小知识**：Cmd + R 是"编译并运行"——先把你写的代码翻译成电脑能懂的机器语言（编译），再执行（运行）。以后你会经常用到它。

第 5 小步：验证调试工具能用（强烈建议现在就做）

1. 在 main.cpp 里，找到 return 0; 那一行，**点击它左边灰色行号的位置**，会出现一个蓝色箭头（这就是"断点"）
2. 再按 Cmd + R，程序会**停在断点那一行**，不再往下执行
3. 观察底部调试区：左侧会列出当前所有变量的值
4. 点调试区顶部的"继续"按钮（▶️ 图标），程序跑完
5. 把蓝色箭头拖走，取消断点

> 断点调试是学编程最重要的技能。**从今天开始养成习惯：程序结果不对 → 先打断点看变量，而不是瞎猜。**

第 1 步完成！进入下一步前，先自己把 Hello World 运行了 3 次（改一改里面的文字再运行，看看变化）。

第 2 步：逐行看懂 C++ 程序结构

打开 Xcode 里的 main.cpp，你会看到类似这样的代码。我们逐行拆开讲：

```
#include <iostream>   // 第 1 行：引入"头文件"
using namespace std;  // 第 2 行：打开"命名空间"
int main() {          // 第 3 行：主函数（程序入口）
    cout << "Hello, World!" << endl;   // 第 4 行：输出一句话
    return 0;         // 第 5 行：告诉系统"正常结束"
}                     // 第 6 行：主函数结束
```

逐行解释：

行	作用	打个比方
#include <iostream>	引入输入输出工具库，cout/cin 都来自这里	相当于"自带工具箱"：先声明"我要用工具箱里的工具"
using namespace std;	允许直接写 cout，不用每次写 std::cout	相当于"我默认用标准牌的工具，不用每次说全名"
int main() {	定义主函数，程序**从这里开始**执行	相当于文章的"正文开头"
cout << "..." << endl;	向屏幕输出文字，endl 表示换行	相当于"大声念出一句话"
return 0;	程序正常结束，返回 0 给操作系统	相当于"鞠躬退场，表示一切正常"

}	结束 main 函数	正文结束

记忆口诀：引头文件、开命名空间、进主函数、写语句、返回 0。

几个马上要懂的细节：

- 每一行"指令"结尾都要有**分号 `;`**，像中文的句号
- // 后面是**注释**，是写给"人"看的说明，电脑会忽略它。多写注释是好习惯！
- 大括号 { } 成对出现，表示代码的"范围"

现在动手：在 main.cpp 里把 "Hello, World!" 改成你自己的名字，运行看看。再删掉一个分号运行，观察 Xcode 报的红色错误——**故意弄错一次，比背十遍都记得牢**。

第 3 步：cout 输出——让程序"说话"

cout 是 C++ 里最常用的输出工具。完整写法：

```
cout << "要输出的内容" << endl;
```

拆开理解：

- cout：输出工具的名字（读作 "see-out"）
- <<：把右边的东西"送进"cout，**箭头指向屏幕**
- "..."：双引号包起来的是**原样输出的文字**
- endl：end line 的缩写，换行

可以输出三种东西：

```
cout << "我是文字";                    // 输出文字
cout << 3 + 4;                         // 输出计算结果：7
cout << "我的年龄是" << 18 << "岁";    // 文字和数字混着输出
```

一行可以连续输出多次（用多个 <<）：

```
cout << "姓名：" << "小明" << "，年龄：" << 18 << endl;
```

为什么要用 endl？不加的话，下一句输出会接在同一行后面。试试把 endl 删掉再运行，你立刻就懂了。

马上动手（练习 1）：输出三行自我介绍，例如：

```
大家好！
我叫小明。
我今年 18 岁。
```

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    cout << "大家好！" << endl;
    cout << "我叫小明。" << endl;
    cout << "我今年 18 岁。" << endl;
    return 0;
}
```

第 4 步：变量与基本类型——让程序"记住东西"

4.1 什么是变量？

程序经常需要**记住数据**，比如你的年龄、身高、名字。变量就是一个"带标签的储物盒"：

- 盒子上写着标签（变量名）
- 盒子里放着数据（变量的值）
- 你可以随时**换掉盒子里的东西**（重新赋值）

```
int age = 18;       // 定义一个叫 age 的盒子，里面放 18
age = 19;           // 把盒子里的东西换成 19
cout << age;        // 输出 19
```

语法拆解：类型 变量名 = 初始值;

【动画演示】变量：储物盒里装值、换值（请在学习软件中观看动画）

4.2 5 种基本类型（逐个认识）

类型	存什么	例子	注意
int	整数	int age = 18;	不能存小数
double	小数	double h = 1.75;	也叫浮点数
char	单个字符	char g = 'A';	**必须用单引号**，只能 1 个字符
bool	真假	bool ok = true;	只有 true/false，输出显示 1/0
string	一串文字	string n = "小明";	**用双引号**，需要 #include <string>

逐个举例：

```
#include <iostream>
#include <string>     // 用 string 必须加这一行
using namespace std;

int main() {
    int age = 18;                 // 整数
    double height = 1.75;         // 小数
    char grade = 'A';             // 单个字符（单引号！）
    bool isHappy = true;          // 真假
    string name = "小明";          // 字符串（双引号！）

    cout << "年龄: " << age << endl;
    cout << "身高: " << height << " 米" << endl;
    cout << "等级: " << grade << endl;
    cout << "开心吗: " << isHappy << endl;   // true 会输出 1
    cout << "名字: " << name << endl;
    return 0;
}
```

注意 char 用单引号 'A'，string 用双引号 "小明"——**这是最常见的低级错误之一**，写错会直接编译报错。

4.3 变量命名规则（背下来）

- 只能由**字母、数字、下划线**组成
- **不能以数字开头**：1name 错，name1 对
- 不能和 C++ 关键字重名（如 int、if、for）
- 建议"见名知意"：存年龄用 age，不要用 a
- 常用风格：多个单词用下划线连接，如 my_age

4.4 变量常见错误

```
int a = 3;
a = "hello";    // ❌ 类型不匹配：int 的盒子里不能放字符串
int 3x = 5;     // ❌ 变量名不能以数字开头
int score;      // ⚠ 声明了但没赋值，里面的值是"随机垃圾"
cout << score;  // ⚠ 输出不可预料！记住：变量先赋值再用
```

马上动手（练习 2）：定义变量保存你的名字、年龄、身高、是否喜欢编程，然后一次性输出。

点开看参考答案

```
#include <iostream>
#include <string>
```

```
using namespace std;
int main() {
    string name = "小明";
    int age = 18;
    double height = 1.75;
    bool loveCpp = true;


    cout << "名字: " << name << endl;
    cout << "年龄: " << age << endl;
    cout << "身高: " << height << endl;
    cout << "喜欢编程吗: " << loveCpp << endl;
    return 0;
}
```

第 5 步：cin 输入——让程序"听懂你"

cin 是 C++ 的输入工具，作用正好和 cout 相反：**从键盘读数据，存进变量。**

```
int age;
cin >> age;           // 程序停下来等你敲数字，按回车后存进 age
```

箭头方向记忆法：cout <<（箭头指向屏幕=输出），cin >>（箭头指向变量=输入）。

一次读多个值：

```
int a, b;
cin >> a >> b;        // 输入两个数，用空格或回车隔开都可以
```

完整例子：问名字和年龄，然后打招呼

```
#include <iostream>
#include <string>
using namespace std;


int main() {
    string name;
    int age;


    cout << "请输入你的名字: ";
    cin >> name;


    cout << "请输入你的年龄: ";
    cin >> age;


    cout << "你好, " << name << "! 你今年 " << age << " 岁。" << endl;
    return 0;
}
```

运行体验：按 Cmd + R 后，程序会停住等你输入。**记得先点击底部调试区，让光标聚焦在那里**，然后输入名字、回车，再输入年龄、回车。

> 输入后没反应？90% 是因为光标没点进底部控制台。点一下调试区再输入即可。

马上动手（练习 3）：让程序问你的"出生年份"，然后算出并输出你今年大概多少岁（2026 - 出生年份）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int birthYear;
    cout << "请输入你的出生年份：";
    cin >> birthYear;
    int age = 2026 - birthYear;
    cout << "你今年大约 " << age << " 岁。" << endl;
    return 0;
}
```

第 6 步：运算符——让程序"会算数"

6.1 算术运算符

运算符	含义	例子	结果
+	加	7 + 2	9
-	减	7 - 2	5
*	乘	7 * 2	14
/	除	7 / 2	**3（！）**
%	取余	7 % 2	1
++	自增 1	i++	i 变 i+1
--	自减 1	i--	i 变 i-1

6.2 最经典的坑：整数除法！

```
int a = 7, b = 2;
cout << a / b;      // 输出 3，不是 3.5!
```

为什么？ 两个 int 相除，C++ 默认做"整数除法"，结果还是整数，小数部分被**直接丢掉**（不是四舍五入）。

想得到小数，两种办法：

```
cout << 7.0 / 2;     // 办法一：至少一边写成小数 → 3.5
cout << (double)7 / 2;  // 办法二：强制转换 → 3.5
```

【动画演示】整数除法：小数被丢掉（请在学习软件中观看动画）

6.3 复合赋值运算符（偷懒写法）

写法	等价于	含义
sum += i	sum = sum + i	累加
sum -= i	sum = sum - i	累减
product *= 2	product = product * 2	累乘
n++	n = n + 1	加 1

6.4 比较运算符与逻辑运算符（第 2 天会大量用到，先认识）

运算符	含义	例子
==	相等（注意是两个等号！）	a == b
!=	不相等	a != b
> < >= <=	大小比较	a >= 0
&&	并且（两边都真才真）	a > 0 && a < 10
||	或者（一边真就真）	a == 0 || b == 0
!	取反	!ok

= 是赋值，== 是判断相等。写 if (a = 5) 的人，都掉进过这个坑。

马上动手（练习 4）：写一个简单计算器——输入两个数，分别输出它们的和、差、积、商（商要正确显示小数）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    double a, b;
    cout << "请输入两个数（用空格隔开）：";
    cin >> a >> b;

    cout << a << " + " << b << " = " << a + b << endl;
    cout << a << " - " << b << " = " << a - b << endl;
    cout << a << " * " << b << " = " << a * b << endl;
    cout << a << " / " << b << " = " << a / b << endl;
    return 0;
}
```

第 7 步：综合小项目——温度转换器（把今天全串起来）

现在把"变量 + 输入 + 输出 + 运算"组合成一个完整的小程序：输入摄氏温度，输出华氏温度。

公式：华氏度 = 摄氏度 × 9 ÷ 5 + 32

```
#include <iostream>
using namespace std;

int main() {
    // 1. 定义变量：存摄氏温度
    double celsius;

    // 2. 提示并输入
    cout << "请输入摄氏温度：";
    cin >> celsius;

    // 3. 计算（注意用 9.0 而不是 9，避免整数除法丢小数）
    double fahrenheit = celsius * 9.0 / 5 + 32;

    // 4. 输出结果
    cout << "华氏温度是：" << fahrenheit << " 度" << endl;

    return 0;
}
```

逐行精讲：

- 第 1 行 #include <iostream>：引入输入输出工具
- double celsius;：声明一个小数变量（还没赋值）
- cout << "请输入摄氏温度："：提示用户（不加 endl，让光标停在提示后）
- cin >> celsius;：读取用户输入存入 celsius
- double fahrenheit = ...：计算并存入新变量。**为什么写 `9.0` 不写 `9`？** 因为 celsius * 9 / 5 里如果都是整数会丢小数（第 6.2 节的坑）
- return 0;：正常结束

自己改一改： 把程序改成反向转换（输入华氏度输出摄氏度，公式 C = (F - 32) × 5 ÷ 9）。

第 8 步：断点调试——学会自己找 bug（重点！）

程序出错不可怕，可怕的是不会找错。调试四步法：

第 1 步：打断点

在代码**行号左边**点一下，出现蓝色箭头。程序运行到这里会暂停。

第 2 步：运行到断点

按 Cmd + R。程序停在断点行（该行还没执行）。

第 3 步：单步执行 + 看变量

用调试区顶部的按钮：

- **Continue（继续 ▶️）**：跑到下一个断点或结束
- **Step Over（跳过）**：执行当前行，**不进入**函数内部
- **Step Into（进入）**：**进入**函数内部，一行行看
- **Step Out（跳出）**：从函数里跳出来

左侧"Variables（变量）"面板会实时显示每个变量的当前值——**这就是你找 bug 的放大镜。**

第 4 步：结束调试

点红色 ■ 停止，或按 Cmd + .。

一个必做的调试练习：

在温度转换器里，把断点打在 double fahrenheit = ... 那一行，运行后观察：

1. 输入 25
2. 断点触发，看左侧 celsius 的值是不是 25
3. 单步执行一行，再看 fahrenheit 变成了多少
4. 你会发现：程序的每一步，你都能亲眼看到

调试心法：**结果不对 → 打断点 → 看变量在哪一步变得不符合预期 → 那一步就是 bug 所在。** 以后所有 7 天都会用到这个技能。

分级练习（先自己想 10 分钟，再看答案）

练习 A（基础）：圆的面积

输入半径 r，输出圆面积（面积 = 3.14159 × r × r）。

提示：r 可能是小数，用 double；注意乘法不要用 × 符号，用 *。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    double r;
    cout << "请输入半径: ";
    cin >> r;
    double area = 3.14159 * r * r;
    cout << "圆的面积是: " << area << endl;
    return 0;
}
```

练习 B（进阶）：三位数反转

输入一个三位数（如 123），输出反转后的数（321）。

提示：用 % 取出个位，用 / 去掉个位，拆成百位、十位、个位再重组。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "请输入一个三位数: ";
    cin >> n;

    int ge = n % 10;         // 个位: n 除以 10 的余数
    int shi = n / 10 % 10;   // 十位: 先去掉个位，再取余
    int bai = n / 100;       // 百位: 直接除以 100

    cout << "反转后: " << ge << shi << bai << endl;
    return 0;
}
```

练习 C（挑战）：秒数换算

输入一个秒数（如 3661），输出它是几小时几分几秒（3661 秒 = 1 小时 1 分 1 秒）。

提示：小时 = 秒 / 3600，剩余秒 = 秒 % 3600，分钟 = 剩余秒 / 60，秒 = 剩余秒 % 60。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int total;
    cout << "请输入秒数: ";
```

```
    cin >> total;


    int h = total / 3600;
    int m = total % 3600 / 60;
    int s = total % 60;


    cout << h << " 小时 " << m << " 分 " << s << " 秒" << endl;
    return 0;
}
```

自测（10 题，不看资料 5 分钟完成）

1. cout 和 cin 分别用来做什么？箭头方向各指向哪边？
2. int、double、char、bool、string 分别存什么类型的数据？
3. char 和 string 在引号使用上有什么区别？
4. 7 / 2 的结果是多少？想得到 3.5 应该怎么写？
5. 判断两个数是否相等，应该用 = 还是 ==？
6. n++ 等价于哪条语句？
7. cout << "年龄：" << 18 << endl; 输出什么？
8. 变量名 2name 合法吗？为什么？
9. return 0; 的作用是什么？
10. 程序运行后要输入数据，但打字没反应，最可能的原因是什么？

点开看答案

1. cout 输出（箭头指向屏幕 <<），cin 输入（箭头指向变量 >>）。
2. 整数、小数、单个字符、真假、字符串。
3. char 用单引号（只能 1 个字符），string 用双引号。
4. 结果是 3（整数除法）；写成 7.0 / 2 或 7 / 2.0 得 3.5。
5. ==（= 是赋值）。
6. n = n + 1;
7. 年龄：18（然后换行）。
8. 不合法，变量名不能以数字开头。
9. 告诉操作系统程序正常结束。

10. 光标没点进底部调试区/控制台。

今日坑点汇总（记下来，都是过来人的血泪）

坑	说明	正确做法
忘选 C++ 语言	建成了 Swift 项目，代码全不对	新建项目时 Language 选 C++
字符串用单引号	"小明" 写成 '小明' 编译报错	string 用双引号
整数除法丢小数	7/2 得 3	至少一边写成小数
= 当 == 用	判断写成赋值，恒为真	判断相等用 ==
变量没赋值就输出	输出随机垃圾值	先赋值再使用
忘了写分号	编译报错	每条语句结尾 ;
运行时输入没反应	光标不在控制台	先点击底部调试区

今日总结 + 明天预告

今天你完成了最重要的奠基：**环境跑通 + 程序结构 + 变量 + 输入输出 + 运算符 + 调试**。

这些是每一行 C++ 代码的"砖块"，接下来就是学习怎么用它们搭出会"思考"的程序。

明天（第 2 天）学什么？ 让程序**自己做决定**（if/switch）和**重复干活**（循环），并做出经典的"猜数字"游戏。

睡前打卡： 回顾今天踩过的坑，在 Xcode 里把温度转换器**默写**一遍（不看答案）。

第 2 天：流程控制——让程序会判断、会重复

预计用时：4～5 小时 ｜ 昨天学会了"输入 → 处理 → 输出"的直筒程序。

今天让程序拥有"大脑"：能根据条件做选择，能反复执行同一段代码。

今天学完，你能做到（打卡清单）

- [ ] 用 if / else if / else 做多条件判断
- [ ] 会用比较运算符和逻辑运算符组合复杂条件
- [ ] 用 switch 处理"多个固定选项"的分支
- [ ] 用 for 循环做固定次数的重复
- [ ] 用 while、do-while 做条件循环
- [ ] 会用 break、continue 控制循环流程
- [ ] 独立写出"猜数字"游戏

今天的路线图

```
第 1 步 回顾 + 为什么要"流程控制"
  ↓
第 2 步 if 判断（单选一）→ if-else → 多条件 else if
  ↓
第 3 步 复杂条件：比较运算符 + 逻辑运算符
  ↓
第 4 步 switch 多分支
  ↓
第 5 步 while 循环（条件循环）
  ↓
第 6 步 for 循环（次数循环）+ 循环嵌套
  ↓
第 7 步 do-while + break/continue
  ↓
第 8 步 综合项目：猜数字游戏（逐步构建）
```

学完分支立刻做判断练习，学完循环立刻做循环练习，最后才做猜数字。

第 1 步：回顾昨天 + 为什么要学"流程控制"

昨天写的程序是**直线型**的：从上到下一条路走到底。

```
cin >> score;            // 读入成绩
cout << score;           // 输出成绩
return 0;                // 结束
```

但真实程序需要**岔路口**和**回路**：

- 成绩 ⩾ 60 说"及格"，否则说"不及格" —— 这是**判断**
- 把 1 到 100 全部加起来 —— 这是**循环**

今天学的 if 和 for 等，就是用来搭"岔路口"和"回路"的零件。

第 2 步：if 判断——让程序做选择

2.1 最简单的 if（满足才执行）

```
if (score >= 60) {
    cout << "及格" << endl;
}
```

执行过程（画一条线在脑子里）：

1. 先算括号里的条件：score >= 60 是真还是假？
2. 条件为真 → 执行大括号里的代码
3. 条件为假 → 跳过，什么都不做

```
int score = 80;
if (score >= 60) {          // 80 >= 60 为真
    cout << "及格" << endl;  // 执行这行 → 输出"及格"
}
```

2.2 if-else（二选一）

```
if (score >= 60) {
    cout << "及格" << endl;
} else {
    cout << "不及格" << endl;
}
```

关键理解：if 和 else **只会走一边**，永远不会两个都执行。就像过安检：符合条件走这边，不符合走那边。

【动画演示】if 判断：条件为真走「是」分支（请在学习软件中观看动画）

2.3 if-else if-else（多选一）

```
if (score >= 90) {
    cout << "优秀" << endl;
} else if (score >= 70) {
    cout << "良好" << endl;
} else if (score >= 60) {
    cout << "及格" << endl;
} else {
    cout << "不及格" << endl;
}
```

执行规则（重要）：从上往下，**第一个条件成立的那个分支执行，后面的都不再看。**

> 所以条件的顺序很重要！如果把 score >= 60 写在最前面，那么 95 分也会先命中"及格"而不是"优秀"——因为 95 >= 60 已经成立了。**顺序 = 从严格到宽松。**

2.4 常见误区

```
if (score >= 60); {          // ❌ if 后面多了分号！{} 变成普通代码块，永远执行
    cout << "及格" << endl;
}


if (score = 60) { ... }      // ❌ 一个等号是赋值！赋值结果是 60（非 0 为真），恒成立
if (score == 60) { ... }     // ✅ 两个等号才是判断相等
```

马上动手（练习 1）：输入一个整数，判断它是**偶数**还是**奇数**（偶数 = 能被 2 整除 = n % 2 == 0）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "请输入一个整数: ";
    cin >> n;
    if (n % 2 == 0) {
        cout << n << " 是偶数" << endl;
    } else {
        cout << n << " 是奇数" << endl;
    }
    return 0;
}
```

第 3 步：复杂条件——比较运算符 + 逻辑运算符

有时一个条件不够，需要组合。

比较运算符（结果都是"真/假"）：

运算符	含义	例子
==	等于	a == b
!=	不等于	a != b
> <	大于 / 小于	a > 0
>= <=	大于等于 / 小于等于	a >= 0

逻辑运算符（组合多个条件）：

运算符	读法	规则	例子
&&	并且	**两边都真**才真	age >= 18 && age <= 60
||	或者	**一边真**就真	score < 60 || score > 100
!	非	真变假、假变真	!ok

完整例子：判断一个年份是否是闰年

规则：能被 4 整除**且**不能被 100 整除，**或者**能被 400 整除。

```
int year = 2024;
bool isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
if (isLeap) {
    cout << year << " 是闰年" << endl;
} else {
    cout << year << " 不是闰年" << endl;
}
```

逐行理解：

- year % 4 == 0：能被 4 整除吗？（2024 能 → 真）
- year % 100 != 0：不能被 100 整除吗？（2024 能 → 真）
- &&：两边都真 → 真
- ||：左边已经真了 → 整体真 → 2024 是闰年

马上动手（练习 2）：输入三个数，判断是否**能构成三角形**（任意两边之和大于第三边）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    double a, b, c;
    cout << "请输入三条边（空格隔开）：";
    cin >> a >> b >> c;


    if (a + b > c && a + c > b && b + c > a) {
        cout << "能构成三角形" << endl;
    } else {
        cout << "不能构成三角形" << endl;
    }
    return 0;
}
```

第 4 步：switch——处理"多个固定选项"

当判断的对象是**一个变量的多个固定取值**时，用 switch 更清晰。

```
int week;
cout << "输入星期几（1-7）：";
cin >> week;


switch (week) {
    case 1: cout << "星期一" << endl; break;
    case 2: cout << "星期二" << endl; break;
    case 3: cout << "星期三" << endl; break;
    case 4: cout << "星期四" << endl; break;
    case 5: cout << "星期五" << endl; break;
    case 6: cout << "星期六" << endl; break;
    case 7: cout << "星期日" << endl; break;
    default: cout << "输入无效" << endl; break;
}
```

语法要点：

- case 值: 后面是"当变量等于这个值时要做的"
- **每个 case 结尾必须有 `break;`**，否则会"穿透"（fall-through）继续执行下一个 case 的代码——这是 switch 最大的坑
- default：所有 case 都不匹配时执行，相当于 else

穿透演示（故意去掉 break 看效果）：

```
int n = 2;
switch (n) {
```

```
    case 1: cout << "一";
    case 2: cout << "二";       // 从这开始
    case 3: cout << "三";       // 没有 break，一路执行到这里
}
// 输出：二三
```

switch vs if 怎么选？ 判断的是"同一个变量的具体取值"→ switch；判断的是"范围"（如 ⩾60）或"多个条件组合" → if。

马上动手（练习 3）：输入 1~5 的一个数，输出对应的"第几季度/阶段"（1=入门, 2=基础, 3=进阶, 4=熟练, 5=精通），超出范围提示无效。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int level;
    cout << "输入你的等级（1–5）：";
    cin >> level;
    switch (level) {
        case 1: cout << "入门" << endl; break;
        case 2: cout << "基础" << endl; break;
        case 3: cout << "进阶" << endl; break;
        case 4: cout << "熟练" << endl; break;
        case 5: cout << "精通" << endl; break;
        default: cout << "无效等级" << endl; break;
    }
    return 0;
}
```

第 5 步：while 循环——只要条件成立就一直做

问题：把 1 加到 100，怎么写？ 复制粘贴 100 行显然不行。循环就是答案。

```
int sum = 0;       // 放结果的"存钱罐"，先放 0
int i = 1;         // 从 1 开始
while (i <= 100) {          // 只要 i 不超过 100
    sum += i;               // 把 i 加进存钱罐
    i++;                    // i 加 1（必须有！否则死循环）
}
cout << "1+2+...+100 = " << sum << endl;   // 5050
```

执行过程（关键）：

1. 判断 i <= 100 是真吗?
2. 真 → 执行大括号里的两行（sum += i; i++;）→ 回到第 1 步
3. 假 → 跳出循环，继续往下

```
i=1: 判断真 → sum=1, i=2
i=2: 判断真 → sum=3, i=3
i=3: 判断真 → sum=6, i=4
...
i=100: 判断真 → sum=5050, i=101
i=101: 判断假 → 跳出 ✅
```

死循环（必踩的坑）：如果忘记写 i++，i 永远等于 1，条件永远成立，程序永远停不下来。

```
int i = 1;
while (i <= 100) {   // ❌ 没有 i++，死循环！
    sum += i;
}
```

遇到程序"卡死不动"，第一反应：**是不是死循环了？** 按 Cmd + . 或点 ■ 停止程序。

while 适合什么场景？ 不知道要循环多少次，只知道"什么时候停"（比如一直输入直到输入 -1 为止）。

马上动手（练习 4）：用 while 循环，让用户不断输入数字并累加，**直到输入 0 为止**，最后输出总和。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0;
    cout << "输入数字累加（输入 0 结束）: " << endl;
    cin >> n;
    while (n != 0) {
        sum += n;
        cin >> n;          // 再读下一个
    }
    cout << "总和 = " << sum << endl;
    return 0;
}
```

第 6 步：for 循环——知道次数时用它

for 循环把"起点、条件、步长"写在一行，比 while 更紧凑：

```
for (int i = 1; i <= 100; i++) {
    sum += i;
}
```

拆解三部分（用分号隔开）：

- int i = 1：**起点**（循环前执行一次）
- i <= 100：**条件**（每次循环前判断，真就继续）
- i++：**步长**（每次循环体执行完执行）

执行顺序：起点 → 判断条件 → 执行循环体 → 步长 → 判断条件 → … → 条件为假跳出

```
i=1 → 判断(1<=100 真) → 执行 → i=2 → 判断(2<=100 真) → 执行 → ... → i=101 → 判断(假) → 跳出
```

三种写法对比（效果一样）：

```
// while 版
int i = 1;
while (i <= 100) { sum += i; i++; }


// for 版（最推荐，结构清晰）
for (int i = 1; i <= 100; i++) { sum += i; }


// for 另一种常见写法：从 0 开始
for (int i = 0; i < 100; i++) { ... }   // 循环 100 次，i 取 0~99
```

for (int i = 0; i < n; i++) 是 C++ 程序员**最常用**的循环写法，见到要秒懂：循环 n 次。

【动画演示】for 循环：指针移动，sum 逐次累加（请在学习软件中观看动画）

6.1 循环嵌套：九九乘法表

循环里再套循环，外层管"行"，内层管"列"：

```
for (int i = 1; i <= 9; i++) {          // 外层：第几行
    for (int j = 1; j <= i; j++) {      // 内层：这一行有几列（到 i 为止）
        cout << j << "x" << i << "=" << i * j << "\t";
    }
    cout << endl;                        // 每行结束换行
}
```

执行过程（想象 i=2 时）：

- 外层进入 i=2
- 内层 j=1：输出 1x2=2
- 内层 j=2：输出 2x2=4
- 内层 j=3：3 <= 2 假 → 内层结束
- cout << endl 换行
- 外层进入 i=3 …

结果长这样：

```
1x1=1
1x2=2  2x2=4
1x3=3  2x3=6  3x3=9
...
```

马上动手（练习 5）：用 for 循环输出 100 以内的**所有偶数**（每行 5 个）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int count = 0;
    for (int i = 2; i <= 100; i += 2) {    // 直接从 2 开始，每次加 2
        cout << i << " ";
        count++;
        if (count % 5 == 0) cout << endl;  // 每 5 个换行
    }
    cout << endl;
    return 0;
}
```

第 7 步：do-while 和 break/continue

7.1 do-while：至少执行一次

普通 while 是"先判断再执行"；do-while 是"**先执行再判断**"——**至少执行一次。**

```
int guess;
do {
    cout << "请输入密码：";
    cin >> guess;
} while (guess != 1234);       // 不对就再问
```

和 while 的区别：

场景	while	do-while
先判断后执行		
先执行后判断		
最少执行次数	0 次	**1 次**

什么时候用 do-while？**"至少要问一次"的场景**，比如猜数字、输密码、输入选项。

7.2 break：立即跳出循环

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;         // i 到 5 就立刻跳出整个循环
    cout << i << " ";
```

```
}
// 输出：1 2 3 4（5 都没输出就跳出了）
```

7.3 continue：跳过本次，继续下一次

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;     // i=3 时跳过本次剩余代码
    cout << i << " ";
}
// 输出：1 2 4 5（3 被跳过，但循环继续）
```

break vs continue 对比：

- break：**结束**整个循环（离开循环）
- continue：**跳过**当前这一次，进入下一次循环

马上动手（练习 6）：输出 1~20 中所有**不是 3 的倍数**的数（提示：用 continue 跳过 3 的倍数）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 20; i++) {
        if (i % 3 == 0) continue;   // 是 3 的倍数就跳过
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

第 8 步：综合项目——猜数字游戏（逐步构建）

把今天学的**随机数 + do-while + if 判断**全部用上。分三步搭：

第 1 小步：生成随机数

```
#include <cstdlib>   // rand, srand
#include <ctime>     // time
srand((unsigned)time(nullptr));   // 用当前时间做"随机种子"
int target = rand() % 100 + 1;    // 生成 1~100 的随机数
```

- srand(...)：给随机数生成器一个"种子"。**不设置的话，每次运行随机数都一样**
- rand() % 100：得到 0~99 的随机数，+1 后变成 1~100

第 2 小步：循环 + 判断（核心逻辑）

```
int guess = 0;
do {
```

```
    cin >> guess;                    // 读入猜测
    if (guess > target) {
        cout << "大了！再试：";
    } else if (guess < target) {
        cout << "小了！再试：";
    }
} while (guess != target);           // 没猜中就一直循环
```

第 3 小步：加上统计，凑成完整程序

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;


int main() {
    srand((unsigned)time(nullptr));
    int target = rand() % 100 + 1;
    int guess = 0, times = 0;


    cout << "【猜数字】我心里想了一个 1~100 的数，你来猜：" << endl;
    do {
        cin >> guess;
        times++;
        if (guess > target) {
            cout << "大了！再试：";
        } else if (guess < target) {
            cout << "小了！再试：";
        }
    } while (guess != target);


    cout << "恭喜猜对！你一共猜了 " << times << " 次。" << endl;
    return 0;
}
```

逐行精讲：

- `int target = rand() % 100 + 1;`：随机目标数
- `do { ... } while (guess != target);`：**先让你猜一次**，不对再循环——完美契合 do-while 的"至少执行一次"
- `times++;`：每猜一次计数 +1
- 程序结束前把 target 输出出来，方便测试（正式版可去掉）

自己改一改（挑战）：

1. 把随机范围改成 1~1000

2. 限制最多猜 10 次，超过就输出"你输了，答案是 X"（提示：用 if (times == 10) break）

3. 玩家猜对后，根据次数给出评价："天才！"（≤5 次）/ "不错"（≤10 次）/ "继续加油"

分级练习

练习 A（基础）：判断闰年

输入年份，输出是否闰年（规则见第 3 步）。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int year;
    cout << "请输入年份：";
    cin >> year;
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
        cout << year << " 是闰年" << endl;
    } else {
        cout << year << " 不是闰年" << endl;
    }
    return 0;
}
```

练习 B（进阶）：打印菱形

输入行数 n（奇数），用 * 打印一个菱形。提示：把菱形分成上半部分（空格递减、星号递增）和下半部分（对称）。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int n = 5;   // 总行数（奇数）
    int mid = n / 2;
    for (int i = 0; i < n; i++) {
        int spaces = i <= mid ? mid - i : i - mid;
        int stars = i <= mid ? 2 * i + 1 : 2 * (n - 1 - i) + 1;
        for (int s = 0; s < spaces; s++) cout << " ";
        for (int k = 0; k < stars; k++) cout << "*";
        cout << endl;
    }
    return 0;
}
```

练习 C（挑战）：质数判断

输入一个大于 1 的整数，判断它是不是质数（只能被 1 和它自己整除）。提示：从 2 试到 n-1，只要有能整除的就不是质数；可以进一步优化为试到 √n。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "请输入一个大于 1 的整数: ";
    cin >> n;


    bool prime = true;
    for (int i = 2; i < n; i++) {
        if (n % i == 0) { prime = false; break; }   // 找到因子，不是质数
    }
    if (prime) {
        cout << n << " 是质数" << endl;
    } else {
        cout << n << " 不是质数" << endl;
    }
    return 0;
}
```

自测（10 题）

1. if (score >= 90) 和 else if (score >= 70) 的顺序能不能互换？为什么？
2. if (a = 5) 有什么问题？
3. switch 里忘记写 break 会发生什么？
4. for (int i = 0; i < 5; i++) 循环体会执行几次？i 最后的值是多少？
5. while 和 do-while 的最大区别是什么？
6. break 和 continue 的区别？
7. 死循环是怎么产生的？怎么在 Xcode 里停止？
8. n % 2 == 0 判断的是什么？
9. 写出用 for 循环求 1~100 奇数和的关键代码。
10. rand() % 100 + 1 生成的范围是什么？

点开看答案

1. 不能互换。从上往下第一个成立的分支执行，90 分也会先命中 70 的条件。

2. 一个等号是赋值，不是判断；赋值后恒为真（非 0）。

3. 会发生"穿透"，继续执行后面 case 的代码。

4. 执行 5 次；循环结束后 i 的值是 5。

5. do-while 至少执行一次（先执行后判断），while 可能一次都不执行。

6. break 结束整个循环；continue 跳过本次剩余代码进入下一次。

7. 循环条件永远为真（如忘了 i++）；按 Cmd + . 或点 ■ 停止。

8. 判断 n 是否是偶数（能被 2 整除）。

9. for (int i = 1; i <= 100; i++) if (i % 2 == 1) sum += i;

10. 1~100。

今日坑点汇总

坑	说明	正确做法
if 后面加分号	if (x) ; 导致分支失效	if 括号后直接跟 {
用 = 代替 ==	判断变赋值，恒为真	判断用 ==
switch 漏 break	穿透执行多个 case	每个 case 末尾加 break
while 忘写条件更新	死循环卡死	循环体内改变条件变量
条件顺序搞反	宽松条件挡在前面	从严格到宽松排列
rand() 不设种子	每次运行随机数相同	先 srand(time(nullptr))

今日总结 + 明天预告

今天你的程序会"思考"了：能判断（if/switch）、能重复（while/for/do-while），还做出了猜数字游戏。**分支 + 循环 + 变量**这三板斧，已经能解决很多实际问题。

明天（第 3 天）学什么？ 把代码"打包"成**函数**（避免重复写）、处理**一批数据**（数组）和**文本**（string），并学会递归。

睡前打卡： 不看答案默写"猜数字"游戏，并给自己加一个"最多猜 10 次"的限制。

第 3 天：函数、数组、字符串

预计用时：5～6 小时 ｜ 前两天写的程序都在 main 里"一锅炖"，代码一多就乱。

今天学习把代码**打包复用**（函数）、处理**一批数据**（数组）、处理**文本**（字符串）。

今天学完，你能做到（打卡清单）

- [ ] 定义和调用函数，理解参数与返回值
- [ ] 理解**值传递**：函数里改参数不会影响外面
- [ ] 会用默认参数和函数重载
- [ ] 理解递归，能写出阶乘和斐波那契
- [ ] 会用一维/二维数组，能写出冒泡排序
- [ ] 会用 string 常用操作和 getline 读取整行
- [ ] 会用断点单步跟踪递归的执行过程

今天的路线图

```
第 1 步 回顾 + 为什么要函数（先感受"重复代码"的痛）
  ↓
第 2 步 函数：定义、调用、参数、返回值（核心）
  ↓
第 3 步 值传递 vs 修改实参（重点概念）
  ↓
第 4 步 默认参数 + 函数重载
  ↓
第 5 步 作用域（局部 vs 全局）
  ↓
第 6 步 递归（自己调用自己，配断点跟踪）
  ↓
第 7 步 数组（一批数据）→ 一维/二维/越界
  ↓
第 8 步 冒泡排序（经典算法，逐步推演）
```

```
  ↓
第 9 步 string 字符串操作 + getline
```

第 1 步：为什么要学函数?

假设你要在程序里三次输出"欢迎光临本系统"，于是复制粘贴了三次：

```
cout << "==========" << endl;
cout << "欢迎光临本系统" << endl;
cout << "==========" << endl;
// ... 200 行代码 ...
cout << "==========" << endl;
cout << "欢迎光临本系统" << endl;
cout << "==========" << endl;
```

问题来了： 哪天想把分隔线从 = 改成 -，你得改 3 处；复制 10 次就得改 10 处，漏改一处就出 bug。

函数就是来解决这个的：把一段代码**打包成一个"工具"**，起个名字，想用就调用一次。

```
void welcome() {
    cout << "==========" << endl;
    cout << "欢迎光临本系统" << endl;
    cout << "==========" << endl;
}


int main() {
    welcome();   // 用一次
    // ... 200 行代码 ...
    welcome();   // 再用一次
    return 0;
}
```

以后想改分隔线，**只改函数里的 3 行**，所有调用处自动生效。

第 2 步：函数核心——定义、调用、参数、返回值

2.1 函数的结构

```
返回类型 函数名(参数列表) {
  函数体
    return 返回值;
}
```

看两个例子：

```
// 例子 1：无参数、无返回值
void sayHello() {
    cout << "你好！" << endl;
}


// 例子 2：两个参数、有返回值（返回两个数中较大的）
int max2(int a, int b) {
    if (a > b) return a;
    else return b;
}
```

2.2 调用函数

```
sayHello();                // 直接调用（无返回值）
int m = max2(3, 7);        // 调用并把返回值存进 m
cout << max2(10, 2);       // 也可以直接用
```

调用过程（脑子里的画面）：

1. 程序跳到函数定义处
2. 把实参（3 和 7）复制给形参（a 和 b）
3. 执行函数体
4. return 把结果带回调用处，程序回到调用点继续往下

2.3 return 的两种角色

```
int max2(int a, int b) {
    if (a > b) return a;   // 返回"值"并结束函数
    return b;              // 走到这说明 a <= b
}
```

- return 值;：既**结束函数**，又**把值送回调用处**
- void 函数里的 return;：只结束函数，不带值（可省略）

2.4 常见错误

```
int max2(int a, int b) {        // 函数声明返回 int
    if (a > b) return a;        // ✅
    // 如果 a <= b 走到函数末尾，没有 return → ❌ 编译警告/未定义行为
}
// 修复：保证所有路径都有返回值（见 2.3 的写法）
```

马上动手（练习 1）：写一个函数 bool isEven(int n)，返回 n 是否为偶数；在 main 里调用并测试 4 和 7。

点开看参考答案

```
#include <iostream>
using namespace std;


bool isEven(int n) {
    return n % 2 == 0;      // 表达式本身就是"真/假"
}


int main() {
    cout << isEven(4) << endl;    // 1（真）
    cout << isEven(7) << endl;    // 0（假）
    return 0;
}
```

大白话：函数拿到的是资料的**复印件**，在复印件上乱涂乱画，原件毫发无损。

第 3 步：值传递——函数里改了外面不变（重点！）

看这段代码，猜猜输出什么：

```
void tryChange(int x) {
    x = 999;
}


int main() {
    int num = 5;
    tryChange(num);
    cout << num;      // 输出 5 还是 999?
    return 0;
}
```

答案是 5。 因为 C++ 默认是**值传递**：调用时把 num 的**值复制一份**给 x，函数里改的是"副本"，外面的 num 毫发无损。

```
main 里的 num（5）  ——复制——▶️ 函数里的 x（5 → 改成 999）
外面的 num 还是 5
```

这个特性重要在哪？

- 好处：函数不会"意外"修改你的数据，安全
- 限制：想通过函数**真正修改外面的变量**，光靠值传递做不到——这正是**明天（第 4 天）指针/引用**要解决的

马上动手（练习 2）：写一个函数尝试交换两个数（用值传递），在 main 里调用后打印，确认"没换成功"，亲身体验这个现象。

点开看参考答案

```cpp
#include <iostream>
using namespace std;


void trySwap(int a, int b) {   // 值传递：a、b 是副本
    int t = a;
    a = b;
    b = t;
}


int main() {
    int x = 1, y = 2;
    trySwap(x, y);
    cout << "x = " << x << ", y = " << y << endl;   // 还是 1 和 2
    return 0;
}
```

【动画演示】值传递：函数改的是副本，外面不变（请在学习软件中观看动画）

第 4 步：默认参数 + 函数重载

4.1 默认参数：不传就用默认值

```cpp
void greet(string name, string prefix = "你好") {
    cout << prefix << ", " << name << "! " << endl;
}


greet("小明");               // 不传 prefix → 用默认值 → "你好，小明！"
greet("小红", "早上好");     // 传了 → "早上好，小红！"
```

规则：默认参数必须放在参数列表**右边**，`void f(int a, int b = 1, int c = 2)` 可以，`void f(int a = 1, int b)` 不行。

4.2 函数重载：同名不同参

同一个名字可以定义多个函数，只要**参数个数或类型不同**，编译器会自动匹配：

```cpp
void greet(string name) {             // 版本 1：一个参数
    cout << "你好，" << name << endl;
}
void greet(string name, int times) { // 版本 2：两个参数，打印多次
    for (int i = 0; i < times; i++) {
        cout << "Hi, " << name << " ";
    }
    cout << endl;
}
```

```
greet("小明");        // 调版本 1
greet("小李", 3);     // 调版本 2
```

只有返回值不同不算重载（编译器无法区分），必须参数不同。

第 5 步：作用域——变量"活"在哪

大括号 `{ }` 决定变量的可见范围：

```
int main() {
    int a = 10;              // a 从这行到 main 结束都可见

    {
        int b = 20;          // b 只在这个花括号里可见
        cout << a + b;       // ✅ 能访问 a（外层）也能访问 b
    }
    // cout << b;            // ❌ 编译错误：b 已"死亡"
    cout << a;               // ✅ 没问题

    return 0;
}
```

局部变量 vs 全局变量：

```
int global = 100;            // 全局变量：所有函数都能用（尽量少用）

void f() {
    cout << global;          // ✅ 函数里能访问全局
}

int main() {
    int local = 5;           // 局部变量：只在 main 里
    f();
    return 0;
}
```

原则：优先用局部变量和参数传递数据；全局变量多了代码难维护（谁都改它，出了问题难定位）。

大白话：递归像**套娃**——大娃娃里套小娃娃，套到最小的那个（出口）再一层层打开。也像查词典：解释里又有生词，继续查，查到认识的词为止。

第 6 步：递归——函数调用自己

6.1 什么是递归

递归就是函数自己调用自己，适合解决"可以分解成更小同类问题"的场景。

6.2 求阶乘 n!

阶乘的定义：5! = 5 × 4 × 3 × 2 × 1

换个写法：n! = n × (n-1)!——这就是递归的核心："求 n! 的问题，变成了求 (n-1)! 的更小问题"。

```
long long factorial(int n) {
    if (n <= 1) return 1;            // ① 递归出口：n=1 时直接返回 1
    return n * factorial(n - 1);     // ② 递归调用：问题规模变小
}
```

执行过程（factorial(4) 的追踪）：

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * 2 * factorial(1)
             = 4 * 3 * 2 * 1        ← 出口，开始"一层层返回"
             = 24
```

递归必须满足两个条件（缺一不可）：

1. **有出口**：递归到某个值时直接返回，不再调用自己（否则无限递归 → 栈溢出崩溃）
2. **规模变小**：每次调用都更接近出口

【动画演示】递归：一层层下潜到出口，再一层层回推（请在学习软件中观看动画）

6.3 斐波那契数列

数列：1, 1, 2, 3, 5, 8, 13...（每一项 = 前两项之和）

```
int fib(int n) {
    if (n <= 2) return 1;            // 出口：前两项都是 1
    return fib(n - 1) + fib(n - 2);  // 分解成两个更小的子问题
}
```

6.4 用断点"看"递归（强烈推荐）

1. 在 return n * factorial(n - 1); 这行打断点
2. 运行，程序第一次停在这
3. 用 **Step Into** 进入 factorial(n-1)，观察每次进入时 n 的值：4 → 3 → 2 → 1
4. 到 n=1 时走 return 1，然后一层层 **Step Out** 返回，观察返回值怎么乘出来的

> 亲眼看过一次递归的进出过程，你就永远不会忘记它。

马上动手（练习 3）：写一个递归函数 int sumTo(int n)，返回 1+2+...+n 的和（提示：sumTo(n) = n + sumTo(n-1)）。

点开看参考答案

```
#include <iostream>
using namespace std;


int sumTo(int n) {
    if (n <= 1) return 1;          // 出口
    return n + sumTo(n - 1);       // 规模变小
}


int main() {
    cout << sumTo(100) << endl;    // 5050
    return 0;
}
```

第 7 步：数组——存"一批"数据

7.1 为什么需要数组

想存 5 个成绩，用 5 个变量太痛苦：

```
int s1, s2, s3, s4, s5;   // 5 个还行，500 个呢？
```

数组就是**一排带编号的格子**，用"数组名 + 下标"访问：

```
int scores[5] = {88, 92, 76, 85, 90};
//  下标:    0    1    2    3    4


cout << scores[0];   // 88（第 1 个）
cout << scores[3];   // 85（第 4 个）
scores[1] = 95;      // 修改第 2 个
```

7.2 铁律：下标从 0 开始！

```
int arr[5];           // 合法下标：0,1,2,3,4
arr[5] = 10;          // ❌ 越界！访问了不存在的第 6 个
```

越界不报错，但很危险：它可能悄悄改掉内存里别处的数据，导致诡异 bug。写代码时心里默念：**长度为 n 的数组，下标最大 n-1**。

7.3 遍历数组 + 求最值

```
int scores[5] = {88, 92, 76, 85, 90};


// 求和
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
cout << "平均分: " << sum / 5.0 << endl;


// 求最大值
int maxV = scores[0];               // 先假设第一个最大
for (int i = 1; i < 5; i++) {
    if (scores[i] > maxV) {
        maxV = scores[i];           // 发现更大的就更新
    }
}
cout << "最高分: " << maxV << endl;
```

求最值的套路：先拿第 0 个当"冠军"，然后逐个挑战，赢了就换人。

7.4 二维数组（表格）

```
int table[3][4] = {             // 3 行 4 列
    {1, 2, 3, 4},               // 第 0 行
    {5, 6, 7, 8},               // 第 1 行
    {9, 10, 11, 12}             // 第 2 行
};


cout << table[1][2];            // 第 2 行第 3 列 = 7
table[0][0] = 100;              // 修改第 1 行第 1 列
```

读法：table[行][列]，都从 0 开始。

马上动手（练习 4）：输入 5 个整数存入数组，输出它们的**平均值**（保留小数）和**最大值**。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int arr[5];
    for (int i = 0; i < 5; i++) cin >> arr[i];


    int sum = 0, maxV = arr[0];
    for (int i = 0; i < 5; i++) {
        sum += arr[i];
```

```
        if (arr[i] > maxV) maxV = arr[i];
    }
    cout << "平均: " << sum / 5.0 << ", 最大: " << maxV << endl;
    return 0;
}
```

第 8 步：冒泡排序——经典算法逐步推演

问题：把 {5, 2, 9, 1, 7} 从小到大排好。

冒泡排序的思路：相邻两个比较，**大的往后"冒"**。每一轮把当前最大的数送到末尾。

第一轮过程（把最大的 9 冒到末尾）：

```
[5, 2, 9, 1, 7]
 5>2? 是 → 交换 → [2, 5, 9, 1, 7]
 5>9? 否 → 不动 → [2, 5, 9, 1, 7]
 9>1? 是 → 交换 → [2, 5, 1, 9, 7]
 9>7? 是 → 交换 → [2, 5, 1, 7, 9]   ← 9 到位
```

第二轮（范围缩小到前 4 个，把 7 冒到位）：[2, 1, 5, 7, 9]

第三轮：[1, 2, 5, 7, 9] … 直到全部有序。

代码实现：

```
int arr[] = {5, 2, 9, 1, 7};
int len = sizeof(arr) / sizeof(arr[0]);   // 计算元素个数: 20/4 = 5


for (int i = 0; i < len - 1; i++) {            // 外层: 一共需要 len-1 轮
    for (int j = 0; j < len - 1 - i; j++) {    // 内层: 每轮少比较一次（末尾已就位）
        if (arr[j] > arr[j + 1]) {             // 相邻比较，大的往后
            int t = arr[j];                    // 交换三个步骤
            arr[j] = arr[j + 1];
            arr[j + 1] = t;
        }
    }
}
```

逐行精讲：

- sizeof(arr)：整个数组占的字节数（5 个 int = 20 字节）
- sizeof(arr[0])：一个元素占的字节数（4 字节）
- 相除得到元素个数 5
- 外层 i < len - 1：5 个数最多需要 4 轮

- 内层 j < len - 1 - i：第 i 轮后，末尾 i 个数已就位，不用再比
- 交换三连：t = a; a = b; b = t;（必须用临时变量，否则覆盖丢失）

【动画演示】冒泡排序：相邻比较，大的往后冒（请在学习软件中观看动画）

马上动手（练习 5）：把上面代码改成**从大到小**排序（只改一处比较符号）。

点开看答案

把 if (arr[j] > arr[j + 1]) 改成 if (arr[j] < arr[j + 1])（小的往后冒，大的留在前面）。

第 9 步：string 字符串操作

string 是 C++ 处理文本的主力类型，比 C 风格的字符数组好用太多。

9.1 基本操作

```
#include <string>


string s1 = "Hello";
string s2 = "C++";


string s3 = s1 + " " + s2;    // 拼接 → "Hello C++"
s3.length();                   // 长度 → 9
s3[0];                         // 按下标取字符 → 'H'（下标也从 0 开始）
s3[1] = 'a';                   // 修改 → "Hallo C++"
```

9.2 常用方法速查

方法	作用	例子
s.length()	长度	"abc".length() → 3
s.substr(起始, 个数)	取子串	"Hello".substr(1,3) → "ell"
s.find("关键词")	查找，返回位置	"Hello".find("ll") → 2
s.empty()	是否为空	"" .empty() → true
s1 + s2	拼接	"a" + "b" → "ab"

find 的坑：找不到时返回 string::npos（一个很大的数），要用它判断：

```
string s = "I love C++";
size_t pos = s.find("Python");
if (pos == string::npos) {
    cout << "没找到 Python" << endl;
```

```
} else {
    cout << "找到了，位置：" << pos << endl;
}
```

9.3 getline：读取含空格的整行

cin >> 遇到**空格就停**，所以 cin >> s 只能读一个单词。想读整行用 getline：

```
string line;
getline(cin, line);        // 读一整行（含空格）
```

经典坑：cin 和 getline 混用

```
int age;
cin >> age;                // 输入 18 后按回车
string name;
getline(cin, name);        // ❌ 读到的 name 是空的！
```

原因：cin >> age 读走 18，但**回车符还留在缓冲区**；getline 读到这个回车符，以为行结束了。

解决：在中间加 cin.ignore(); 清掉残留的回车符：

```
int age;
cin >> age;
cin.ignore();              // 清掉缓冲区里的换行符
string name;
getline(cin, name);        // ✅ 正常读整行
```

马上动手（练习 6）：输入一句英文（含空格），用程序统计其中**字母 a 出现的次数**。

点开看参考答案

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string line;
    cout << "输入一句英文：";
    getline(cin, line);


    int count = 0;
    for (int i = 0; i < (int)line.length(); i++) {
        if (line[i] == 'a' || line[i] == 'A') count++;
    }
    cout << "字母 a 出现了 " << count << " 次" << endl;
    return 0;
}
```

分级练习

练习 A（基础）：数组逆序输出

输入 5 个整数，逆序输出（最后输入的先输出）。提示：从 len-1 往前遍历到 0。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int arr[5];
    for (int i = 0; i < 5; i++) cin >> arr[i];
    for (int i = 4; i >= 0; i--) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

练习 B（进阶）：二分查找

在**已排序**数组里找数字 x，返回下标，找不到返回 -1。

思路：每次都和中间的数比，决定去左半边还是右半边找，每轮排除一半。

点开看答案

```
#include <iostream>
using namespace std;


int binarySearch(int arr[], int len, int x) {
    int lo = 0, hi = len - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (arr[mid] == x) return mid;
        else if (arr[mid] < x) lo = mid + 1;   // 去右半边
        else hi = mid - 1;                     // 去左半边
    }
    return -1;
}


int main() {
    int arr[] = {1, 3, 5, 7, 9, 11};
    int len = sizeof(arr) / sizeof(arr[0]);
    cout << binarySearch(arr, len, 7) << endl;   // 3
    cout << binarySearch(arr, len, 8) << endl;   // -1
    return 0;
}
```

练习 C（挑战）：字符串反转

输入一个字符串，原地反转后输出。提示：用两个下标，一个从前往后、一个从后往前，交换字符。

点开看答案

```
#include <iostream>
#include <string>
```

```
using namespace std;
int main() {
    string s;
    cout << "输入字符串: ";
    getline(cin, s);


    int left = 0, right = (int)s.length() - 1;
    while (left < right) {
        char t = s[left];
        s[left] = s[right];
        s[right] = t;
        left++;
        right--;
    }
    cout << s << endl;
    return 0;
}
```

自测（10 题）

1. 值传递时，函数里修改参数，外面的变量会变吗？为什么？
2. void 函数需要 return 吗？
3. 默认参数必须放在参数列表的哪一边？
4. 函数重载靠什么区分？
5. 递归必须满足哪两个条件？
6. 数组长度为 5，合法下标范围是？
7. sizeof(arr) / sizeof(arr[0]) 用来算什么？
8. s.find("x") 找不到时返回什么？
9. cin >> 和 getline 有什么区别？混用会出什么问题？
10. 冒泡排序每轮结束后，有什么"规律"？

点开看答案

1. 不会。传进去的是值副本，函数改的是副本。
2. 不需要，可写可不写 return;。
3. 右边。
4. 参数个数或参数类型不同。
5. 有出口 + 问题规模越来越小。
6. 0 到 4。

7. 数组的元素个数。
8. string::npos。
9. cin 遇空格停，getline 读整行；混用要用 cin.ignore() 清掉残留换行符。
10. 每一轮把当前最大值"冒"到末尾，下一轮范围缩小 1。

今日坑点汇总

坑	说明	正确做法
数组越界	arr[n] 访问不存在的位置	记住下标 0~n-1
忘写递归出口	无限递归 → 栈溢出崩溃	先写 if (n <= ...) return ...
cin 和 getline 混用	getline 读到空行	中间加 cin.ignore()
函数所有路径没 return	返回垃圾值	保证每个分支都有 return
默认参数放左边	编译错误	默认参数放参数列表右边
find 返回值直接当下标	找不到时是超大数	先判断 == string::npos

今日总结 + 明天预告

今天你的代码开始"有结构"了：函数打包、数组批量处理、字符串文本处理、递归思维，还有冒泡排序和二分查找两个经典算法。

明天（第 4 天）学什么？ 进入 C++ 最核心也最硬核的部分——**指针、引用、动态内存**。这是 C++ 和 Python/Java 最大的区别，也是很多人的"劝退点"。别怕，明天会用"信箱"的比喻和大量画图，把它彻底讲透。

睡前打卡： 默写冒泡排序，并给阶乘递归打断点，亲手"看"一次递归进出。

第 4 天：指针、引用、动态内存（C++ 最关键的一天）

预计用时：5～6 小时（可以拆成两个晚上）｜这是 C++ 和 Python/Java 最大的不同点，也是公认的难点。今天会用"信箱 + 门牌号"的比喻，配大量画图和断点，把它彻底讲透。

慢慢来，值得。

今天学完，你能做到（打卡清单）

- [ ] 理解变量、地址、指针三者的关系（能画图说明）
- [ ] 会用 * 解引用、& 取地址
- [ ] 分清指针与引用，会用引用参数真正修改实参
- [ ] 会用 new / delete 动态分配和释放内存
- [ ] 知道野指针、悬空指针、内存泄漏是什么，并会避免
- [ ] 会用断点观察地址和值的变化

今天的路线图

```
第 1 步 回顾（值传递的痛点）+ 内存的直觉模型
  ↓
第 2 步 地址：& 取地址（信箱门牌号）
  ↓
第 3 步 指针：存地址的变量（* 解引用）
  ↓
第 4 步 空指针 / 野指针（安全红线）
  ↓
第 5 步 指针与数组、指针运算
  ↓
第 6 步 引用：& 别名（比指针友好）
  ↓
第 7 步 值传递 vs 引用传递 vs 指针传递（三兄弟对比）
```

```
  ↓
第 8 步 动态内存：new / delete（堆上的储物柜）
  ↓
第 9 步 内存泄漏 / 悬空指针（两大杀手）
```

前 5 步建立"地址"直觉，后 4 步用上这个直觉。**每一步都画图、都打断点验证。**

第 1 步：回顾 + 内存的直觉模型

昨天留下一个问题：值传递改不了外面的变量。

```
void trySwap(int a, int b) { int t = a; a = b; b = t; }
int x = 1, y = 2;
trySwap(x, y);   // 换失败！因为函数里操作的是副本
```

要解决它，必须先搞懂一个底层问题：**变量到底存在哪里？**

内存的直觉模型（请画下来）：

想象内存是一排**信箱**（或者一排储物柜）：

```
地址（门牌号）  0x100  0x104  0x108  0x10C ...
信箱内容      25      ...    ...    ...
```

- 每个信箱有一个**编号（地址）**
- 每个变量住在某个信箱里，**地址是唯一的**
- int age = 25; 做的事情：找一个信箱，把 25 放进去

核心认知：变量 = 地址 + 值。 平时我们用"变量名"指代它，但底层是"某个地址上的某个值"。

第 2 步：取地址 &——问"你的门牌号是多少"

C++ 用 & 运算符取一个变量的**地址**：

```
int age = 25;


cout << age;     // 输出信箱里的内容：25
cout << &age;    // 输出信箱的门牌号：类似 0x16f1ab6c8
#include <iostream>
using namespace std;
int main() {
    int age = 25;
    int height = 180;
```

```
    cout << "age 的值: " << age << endl;           // 25
    cout << "age 的地址: " << &age << endl;         // 一串十六进制数
    cout << "height 的值: " << height << endl;     // 180
    cout << "height 的地址: " << &height << endl;  // 另一串数

    // 你会发现：相邻变量的地址往往挨得很近，差 4 个字节（一个 int 的大小）
    return 0;
}
```

运行观察：两次运行，地址通常不同（系统每次分配的位置可能不同），但**同一变量在同一运行里地址固定**。这就是"门牌号"。

> 地址的写法 0x... 是十六进制，不用会换算，知道它是"门牌号"就行。

第 3 步：指针——存"门牌号"的变量

> 大白话：指针 = 写着**门牌号**的便利贴。你不用把箱子搬来，拿着门牌号就能找到箱子、改里面的东西。

3.1 定义指针

指针（pointer）就是一个用来存地址的变量：

```
int age = 25;       // 普通变量：存值
int *p = &age;      // 指针变量：存 age 的地址

// 读法：p 是指向 int 的指针（int *p）
```

现在内存里有两个东西：

```
变量 age（信箱）       指针 p（另一个信箱）
+-----------+            +------------------+
| 25        |            | &age（0x16f...） |
+-----------+            +------------------+
 地址 0x16f...             地址 0x16f+8...
```

- p 的值 = age 的地址（门牌号）
- 通过 p，我们能**找到并修改** age

3.2 解引用 *——通过门牌号进去拿东西

```
*p = 30;          // 顺着 p 里存的地址，把那个信箱的内容改成 30
cout << age;      // 输出 30！age 真的被改了
```

指针三件套（必须分清，背下来）：

写法	含义	打个比方
p	指针里存的**地址**	门牌号
*p	地址里存的**值**	敲门进去看到的箱子里的东西
&age	取 age 的**地址**	问 age 的门牌号

【动画演示】指针：p 存地址，*p 修改原变量（请在学习软件中观看动画）

3.3 完整演示 + 断点观察

```
#include <iostream>
using namespace std;
int main() {
    int age = 25;
    int *p = &age;


    cout << "age = " << age << endl;      // 25
    cout << "&age = " << &age << endl;    // 地址
    cout << "p = " << p << endl;          // 和 &age 一样
    cout << "*p = " << *p << endl;        // 25


    *p = 30;                              // 通过指针修改
    cout << "age = " << age << endl;      // 30 ✅


    return 0;
}
```

必做调试：在 *p = 30; 打断点，看变量面板里 age、p、*p 三个值，理解它们的区别。然后单步执行，观察 *p = 30 执行后 age 的变化。

马上动手（练习 1）：定义一个 double price = 9.9; 和指向它的指针，用指针把价格改成 19.9，再输出验证。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    double price = 9.9;
    double *p = &price;       // 指向 double 的指针
    *p = 19.9;                // 通过指针修改
    cout << price << endl;    // 19.9
    return 0;
}
```

第 4 步：空指针与野指针——安全红线

空指针 `nullptr`：不指向任何地方的指针，表示"我还没想好指哪"。

```
int *p = nullptr;       // 安全的"空"
if (p == nullptr) {
    cout << "p 还没指向任何地方" << endl;
}
// 用之前先判断，安全
```

野指针（危险！）：**没有初始化**就使用的指针，里面是随机垃圾地址。

```
int *bad;       // ❌ 没初始化！里面是随机值（野指针）
*bad = 10;      // ❌ 往随机地址写数据 → 大概率崩溃！
```

为什么危险？ 你根本不知道 bad 指向哪——可能是别人的内存、系统内存。写进去轻则崩溃，重则数据损坏。**这是 C++ 最常见、最隐蔽的 bug 来源之一。**

三条铁律（背下来）：

1. 指针**定义时就要初始化**：要么 = &变量，要么 = nullptr
2. 解引用前**先判断是不是空**
3. 指针指向的内存释放后，**立刻置为 nullptr**

马上动手（练习 2）：写出"安全使用指针"的完整流程（定义 → 判断空 → 使用）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int value = 42;
    int *p = &value;        // 1. 定义时就初始化


    if (p != nullptr) {     // 2. 用前判断
        cout << *p << endl; // 3. 使用
    }
    return 0;
}
```

第 5 步：指针与数组、指针运算

5.1 数组名就是首元素地址

```
int arr[] = {10, 20, 30};
int *pa = arr;              // 等价于 int *pa = &arr[0];
```

arr 这个名字本身就代表首元素的地址，所以可以直接赋给指针。

5.2 指针 + 数字 = 跳过几个元素

```
cout << *pa;        // 10（第 0 个）
cout << *(pa + 1);  // 20（第 1 个）
cout << *(pa + 2);  // 30（第 2 个）
```

关键：pa + 1 不是地址加 1，而是**跳过一整个 int（4 字节）**。指针加 n，地址实际加 n × 类型大小。

5.3 用指针遍历数组

```
int arr[] = {10, 20, 30, 40};
int *p = arr;
for (int i = 0; i < 4; i++) {
    cout << *(p + i) << " ";   // 等价于 arr[i]
}
```

这也解释了为什么"函数里改数组会影响外面"：数组传给函数时，传的是**首地址**（不是副本），所以函数通过地址改的就是原数组。

```
void addOne(int arr[], int n) {   // 这里的 arr[] 其实是 int *arr
    for (int i = 0; i < n; i++) {
        arr[i] += 1;              // 改的就是外面的数组！
    }
}
```

马上动手（练习 3）：用指针遍历数组，输出数组的**最大值**。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {7, 3, 9, 1, 5};
    int *p = arr;
    int maxV = *p;                    // 先取第 0 个
    for (int i = 1; i < 5; i++) {
        if (*(p + i) > maxV) maxV = *(p + i);
    }
    cout << "最大值：" << maxV << endl;  // 9
    return 0;
}
```

> 大白话：引用 = **外号**。你叫"小明"也好、叫"小明同学"也好，喊的都是同一个人；改外号指向的内容，改的就是本人。

第 6 步：引用——比指针友好的"别名"

引用（reference）就是给变量起个"别名"：别名和本名指向**同一个信箱**，用哪个都一样。

```
int a = 10;
int &r = a;       // r 是 a 的引用（别名），用 & 定义
r = 20;           // 相当于 a = 20
cout << a;        // 20 ✅
```

引用和指针的本质区别（背下来）：

对比	指针	引用
定义写法	int *p = &a;	int &r = a;
能不能为空	可以是 nullptr	**必须初始化，不能为空**
能不能改指向	可以	**一旦绑定不能改**
取里面的值	要解引用 *p	直接用 r
适合场景	动态内存、遍历等	函数参数、返回多个值

一句话：指针是"门牌号"，可以改；引用是"外号"，天生就是那个人。

> 为什么引用更安全？因为它**永远指向一个真实存在的变量**，没有野引用、空引用的问题。

马上动手（练习 4）：定义一个变量并用引用给它起别名，通过别名修改，验证原变量变了；再试试能不能让引用"改指向"另一个变量（会编译报错吗？）。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 99;
    int &r = a;       // r 绑定 a
    r = 20;           // a 变 20
    cout << "a = " << a << endl;   // 20

    r = b;            // 注意！这不是"改指向"，而是"把 b 的值赋给 a"
    cout << "a = " << a << ", b = " << b << endl;   // 99, 99

    // int &r2;       // ❌ 编译错误：引用必须初始化
    return 0;
}
```

第 7 步：三种传参方式对比——解决"换不掉"的问题

昨天的问题现在可以解决了。三种方式：

```
// 方式 1：值传递（副本，改不了外面）
void swapByValue(int a, int b) {
    int t = a; a = b; b = t;
}


// 方式 2：指针传递（传地址，能改外面）
void swapByPtr(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}


// 方式 3：引用传递（传别名，能改外面，最推荐）
void swapByRef(int &a, int &b) {
    int t = a; a = b; b = t;
}
int x = 1, y = 2;
swapByValue(x, y);   // 失败：x,y 还是 1,2
swapByPtr(&x, &y);   // 成功：要传地址
swapByRef(x, y);     // 成功：直接传变量
```

理解方式 2（指针）：&x 是 x 的门牌号，函数拿到门牌号后，*a 就是顺着门牌号操作 x 本体。

理解方式 3（引用）：int &a 就是"a 是 x 的别名"，函数里所有 a 都是 x。

用引用返回多个结果（函数只能 return 一个值）：

```
void minMax(int arr[], int n, int &minV, int &maxV) {
    minV = arr[0];
    maxV = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < minV) minV = arr[i];
        if (arr[i] > maxV) maxV = arr[i];
    }
}


int nums[] = {7, 3, 9, 1, 5};
int mn, mx;
minMax(nums, 5, mn, mx);   // mn、mx 被"填"上结果
cout << mn << " " << mx;   // 1 9
```

选哪个？ 默认用**引用**（安全、直观）；需要"可能没有值"或动态内存时用指针。

马上动手（练习 5）：写一个函数，用引用参数同时求出"平均值"和"最大值"并返回。

点开看参考答案

```
#include <iostream>
using namespace std;


void stats(int arr[], int n, double &avg, int &maxV) {
```

```
    int sum = 0;
    maxV = arr[0];
    for (int i = 0; i < n; i++) {
        sum += arr[i];
        if (arr[i] > maxV) maxV = arr[i];
    }
    avg = (double)sum / n;
}


int main() {
    int arr[] = {10, 20, 30, 40};
    double avg;
    int maxV;
    stats(arr, 4, avg, maxV);
    cout << "平均" << avg << "，最大" << maxV << endl;  // 25, 40
    return 0;
}
```

第 8 步：动态内存——new 和 delete

8.1 为什么要"动态"内存

之前所有变量都在"栈"上，**大小必须在编译时定死**：

```
int n;
cin >> n;
int arr[n];        // ❌ 部分编译器不允许：数组大小要编译时确定
```

但很多场景**运行时才知道要多大**（用户输入 n）。这时用 new 在**堆**上动态申请：

```
int n;
cout << "要几个数？";
cin >> n;


int *arr = new int[n];    // 在堆上申请 n 个 int（大小运行时才定）
for (int i = 0; i < n; i++) {
    cin >> arr[i];        // 用起来和普通数组一样
}
delete[] arr;             // 用完必须释放！
arr = nullptr;
```

8.2 单个变量的动态分配

```
int *p = new int;      // 申请一个 int 的空间
*p = 99;               // 存值
delete p;              // 释放（注意：不是 delete[]）
p = nullptr;
```

【动画演示】new/delete：申请、使用、释放（请在学习软件中观看动画）

8.3 两条铁律（必须背）

1. **`new` 必须配对 `delete`**（数组用 delete[]），一一对应
2. 释放后**立刻把指针置为 `nullptr`**

```
int *p = new int(5);
delete p;
// 忘记 p = nullptr;  ← 危险!
*p = 99;   // ❌ p 现在是"悬空指针"，指向已释放的内存
```

8.4 为什么必须 delete? ——内存泄漏

程序申请的堆内存，**不会自动释放**（不像栈上变量离开作用域自动销毁）。忘了 delete：

```
while (true) {
    int *p = new int[1000];   // 每次都申请，从不释放
    // ... 用 ...
    // ❌ 忘了 delete[] —— 内存悄悄流失
}
```

跑久了，可用内存越来越少，程序越来越慢直到崩溃——这叫**内存泄漏**。服务器程序最怕这个。

马上动手（练习 6）：输入 n，动态创建 n 个元素的数组，求平均值后释放内存。

点开看参考答案

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "输入个数: ";
    cin >> n;


    int *arr = new int[n];
    int sum = 0;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        sum += arr[i];
    }
    cout << "平均: " << (double)sum / n << endl;


    delete[] arr;
    arr = nullptr;
    return 0;
}
```

分级练习

练习 A（基础）：用指针写 strlen

不用 <cstring>，用指针数出字符串长度（字符串以 '\0' 结尾）。

点开看答案

```
#include <iostream>
using namespace std;


int myStrlen(const char *s) {
    int len = 0;
    while (*s != '\0') {   // 字符串结尾是 '\0'
        len++;
        s++;               // 指针后移一个字符
    }
    return len;
}


int main() {
    const char *msg = "hello";
    cout << myStrlen(msg) << endl;   // 5
    return 0;
}
```

练习 B（进阶）：指针逆序

输入 5 个数，用指针从后往前输出。提示：p 指向首元素，*(p+4) 是最后一个。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
    int arr[5];
    for (int i = 0; i < 5; i++) cin >> arr[i];
    int *p = arr;
    for (int i = 4; i >= 0; i--) {
        cout << *(p + i) << " ";
    }
    cout << endl;
    return 0;
}
```

练习 C（挑战）：动态二维数组

输入行数 r 和列数 c，动态创建 r×c 的二维数组，填数并求和。

提示：int **m = new int*[r];，每行再 new int[c]，用完逐行释放。

点开看答案

```
#include <iostream>
using namespace std;
int main() {
```

```
    int r, c;
    cout << "输入行和列: ";
    cin >> r >> c;


    int **m = new int*[r];        // 先建 r 个"行指针"
    for (int i = 0; i < r; i++) {
        m[i] = new int[c];        // 每行再建 c 个 int
    }


    int sum = 0;
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            m[i][j] = i + j;      // 随便填个数
            sum += m[i][j];
        }
    }
    cout << "总和 = " << sum << endl;


    for (int i = 0; i < r; i++) delete[] m[i];   // 逐行释放
    delete[] m;
    return 0;
}
```

自测（10 题）

1. int *p = &age; 中，p、*p、&age 分别是什么?
2. 指针为什么必须初始化?
3. 引用和指针最大的区别是什么？（至少 2 点）
4. new 出来的内存，不 delete 会怎样?
5. delete 之后为什么要把指针置 nullptr?
6. pa + 1 真的是地址 +1 吗?
7. 数组名 arr 赋给指针，代表什么?
8. 三种传参方式里，哪种最推荐用来修改实参? 为什么?
9. new int[10] 应该用 delete 还是 delete[]?
10. 什么叫"悬空指针"?

点开看答案

1. p 是地址；*p 是地址里的值；&age 是 age 的地址。

2. 不初始化就是野指针，指向随机地址，解引用会崩溃或损坏数据。

3. 引用必须初始化、不能为空、绑定后不能改指向；指针可以 nullptr 且可改指向。

4. 内存泄漏，长期运行内存越占越多直到崩溃。

5. 防止悬空指针——释放后地址仍存在但内容无效，再用会出错。

6. 不是，是跳过一整个类型的字节数。

7. 首元素的地址（&arr[0]）。

8. 引用（安全直观）；指针也行但要小心空指针。

9. delete[]。

10. 指向的内存已被释放、但指针还保留着旧地址的指针。

今日坑点汇总

坑	说明	正确做法
野指针	未初始化就解引用	定义时初始化或置 nullptr
悬空指针	delete 后还用它	释放后立即置 nullptr
内存泄漏	new 了忘 delete	new/delete 一一配对
数组 delete 用错	delete 而非 delete[]	数组用 delete[]
引用不初始化	int &r; 编译错误	定义引用必须同时绑定
指针运算误解	以为 +1 是地址+1	是跳过一整个类型

今日总结 + 明天预告

今天啃下了 C++ 最难啃的骨头：地址、指针、引用、动态内存。只要把"信箱 + 门牌号"的画图理解透，这些概念其实很朴素。从明天开始，我们把数据和操作装进"对象"，进入面向对象编程——那时候引用会非常有用。

明天（第 5 天）学什么？ class（类）与 object（对象）：封装、构造函数、析构函数，并用它做一个"银行账户"。

睡前打卡： 向别人（或空气）完整讲一遍"指针三件套：p、*p、&age"，讲不出来就再看一遍第 3 步。

第 5 天：类与对象——面向对象编程入门

预计用时：5～6 小时 ｜ 前 4 天学的叫"面向过程"：数据是数据、函数是函数。

今天学习"面向对象"：把**数据和操作它的函数打包成一个整体**——类（class）与对象（object）。

今天学完，你能做到（打卡清单）

- [ ] 理解"为什么需要类"，能说出类和对象的区别
- [ ] 定义 class，理解成员变量和成员函数
- [ ] 理解 public / private / protected 三种访问权限
- [ ] 会用构造函数初始化对象、析构函数做清理
- [ ] 理解封装：用 getter / setter 保护数据
- [ ] 会用 this 指针和静态成员
- [ ] 独立设计一个简单的类（如银行账户、圆）

今天的路线图

```
第 1 步 回顾 + 为什么要类（先感受"数据与函数分离"的痛点）
  ↓
第 2 步 struct：最朴素的"打包"
  ↓
第 3 步 class 与访问权限（public/private）
  ↓
第 4 步 构造函数（创建时自动初始化）
  ↓
第 5 步 析构函数（销毁时自动清理）
  ↓
第 6 步 封装：getter / setter
  ↓
第 7 步 this 指针 + 静态成员
  ↓
第 8 步 对象数组 + 综合项目：银行账户
```

第 1 步：为什么要类？

回想第 3 天：我们用一个数组存 5 个成绩，再用一个函数求平均。

```
int scores[5] = {88, 92, 76, 85, 90};
double average(int arr[], int n);   // 函数在"外面"处理数据
```

现在来个更复杂的场景：要管理一个学生，他有**名字、年龄、成绩**，还要能"打印信息""改成绩"。用现在的知识会写成：

```
string name = "小明";
int age = 18;
double score = 88.5;


void printInfo(string n, int a, double s) { ... }
void setScore(double s) { ... }     // 问题：怎么"改"外面的 score？
```

痛点很明显：

1. 数据散落各处，和操作它的函数**没有绑定关系**
2. 函数想改数据，得靠引用参数传进传出，啰嗦
3. 如果有 3 个学生，就得 9 个变量、一堆函数，完全失控

类的解决思路：把"学生的数据"和"学生能做的事"**装进同一个模具**，用模具造出任意多个学生对象：

```
class Student（模具）
├── 数据：name, age, score
└── 行为：printInfo(), setScore()
     ↓ 用模具造
s1（小明）  s2（小红）  s3（小刚）...
```

术语：

- **类（class）**：模具/模板，描述"有什么数据、能做什么"
- **对象（object）**：用模具造出来的具体实例

【动画演示】类与对象：一个模具造出多个实例（请在学习软件中观看动画）

> 大白话：类 = **蛋糕模具**（描述形状），对象 = 用模具做出来的**蛋糕**（具体的一块）。一个模具可以做出无数块蛋糕，每块都有自己的馅料（数据）。

第 2 步：struct——最朴素的"数据打包"

先看 C++ 里最简单的打包工具 struct（结构体）：

```
struct Student {
    string name;
    int age;
    double score;
};
Student s1;               // 造一个学生
s1.name = "小明";          // 直接访问成员（. 点号）
s1.age = 18;
s1.score = 88.5;


cout << s1.name << " " << s1.age << " " << s1.score << endl;
```

和数组对比：数组是一排"同类型"的格子；结构体是把"不同类型"的数据捆在一起。s1.name 用点号访问。

struct 的局限：它只是"数据容器"，没法把"行为（函数）"也带进去（虽然能加，但默认全公开，不安全）。真正强大的是 class。

第 3 步：class 与访问权限

3.1 定义一个类

```
class Student {
public:                         // 公有：外部可以访问/调用
    string name;
    int age;


    void printInfo() {          // 成员函数：和这个类的数据"绑在一起"
        cout << name << " " << age << endl;
    }
};
```

成员函数里可以直接用成员变量（不用传参，因为函数本来就属于这个对象）。

```
Student s1;
s1.name = "小明";
s1.printInfo();       // 输出：小明 18
```

3.2 三种访问权限

关键字	外部能否访问	子类能否访问	用途
public			对外提供的方法
private			内部数据，不让外面乱碰
protected			第 6 天学继承时用

```
class BankAccount {
private:
    double balance;          // 私有：外面碰不到


public:
    void deposit(double m) { balance += m; }   // 公有：通过方法操作
};
BankAccount acc;
// acc.balance = 999;        // ❌ 编译错误：balance 是 private
acc.deposit(100);            // ✅ 通过公有方法操作
```

为什么要把数据设 private？ 防止被乱改。想想：余额可以被随便赋成负数、被清零，银行还不乱套？把数据锁起来，只留"受控的通道"，这就是**封装**的雏形。

> 设计原则：**成员变量默认 private，操作走公有方法。**

第 4 步：构造函数——创建时自动初始化

4.1 问题

```
Student s1;
cout << s1.name;    // ⚠ name 没赋值，输出垃圾
```

每次创建对象后都要手动赋值，容易忘。**构造函数**就是用来解决这个的：**创建对象时自动执行**，负责初始化。

4.2 定义构造函数

```
class Student {
private:
    string name;
    int age;
    double score;


public:
    // 构造函数：名字和类名一样，没有返回值类型
    Student(string n, int a, double s) {
        name = n;
        age = a;
        score = s;
    }


    void printInfo() {
        cout << name << " " << age << " " << score << endl;
    }
};
```

```
Student s1("小明", 18, 88.5);   // 创建时自动调用构造函数，完成初始化
Student s2("小红", 19, 95.0);
s1.printInfo();
```

创建对象时的执行顺序：分配内存 → 执行构造函数 → 对象可用。

4.3 初始化列表（更推荐的写法）

```
Student(string n, int a, double s) : name(n), age(a), score(s) {
    // 函数体里可以什么都不写
}
```

冒号后面的部分就是初始化列表：直接初始化成员。效率和安全性都更好，第 6 天继承时你还会再见到它。

4.4 默认构造函数

如果不写任何构造函数，编译器会生成一个"什么都不做的默认构造函数"。**但一旦你写了带参数的构造函数，默认构造函数就没了**：

```
class Student {
public:
    Student(string n) { ... }
};
Student s1;        // ❌ 编译错误！没有无参构造函数了
Student s2("小明"); // ✅
```

马上动手（练习 1）：设计一个 Circle 类，私有成员 radius，构造函数初始化，公有方法 area() 和 perimeter()。

点开看参考答案

```
#include <iostream>
using namespace std;


class Circle {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}          // 初始化列表


    double area() { return 3.14159 * radius * radius; }
    double perimeter() { return 2 * 3.14159 * radius; }
};


int main() {
    Circle c(5);
    cout << "面积: " << c.area() << endl;       // 约 78.54
    cout << "周长: " << c.perimeter() << endl;  // 约 31.42
    return 0;
}
```

第 5 步：析构函数——销毁时自动清理

析构函数（destructor）和构造函数相反：**对象销毁时自动执行**，用来做清理（比如释放对象内部 new 的内存）。

```
class Student {
public:
    ~Student() {               // 名字是 ~类名，没有参数，没有返回值
        cout << "学生对象被销毁了" << endl;
    }
};
```

什么时候销毁？ 对象离开它所在的花括号作用域时：

```
int main() {
    {
        Student s("小明", 18, 90);
        // 使用 s...
    }                           // ← 到这里 s 销毁，析构函数自动执行
    cout << "main 继续" << endl;
    return 0;
}
// 输出:
// 学生对象被销毁了
// main 继续
```

为什么要学它？ 如果一个类内部用 new 申请了内存，必须在析构函数里 delete，否则内存泄漏。标准写法：

```
class DynamicArray {
private:
    int *data;
    int size;
public:
    DynamicArray(int n) {
        size = n;
        data = new int[n];          // 申请
    }
    ~DynamicArray() {
        delete[] data;              // 释放（析构函数的经典用途）
        data = nullptr;
    }
};
```

马上动手（练习 2）：给 Circle 类加一个析构函数，输出"圆对象已销毁"，创建 2 个对象观察销毁顺序（后创建的先销毁）。

点开看参考答案

```
#include <iostream>
using namespace std;


class Circle {
private:
    double radius;
public:
    Circle(double r) : radius(r) { cout << "创建圆" << endl; }
    ~Circle() { cout << "销毁圆" << endl; }
    double area() { return 3.14159 * radius * radius; }
};


int main() {
    Circle c1(1), c2(2);
    cout << "--- main 即将结束 ---" << endl;
    return 0;
}
// 输出顺序：创建圆 → 创建圆 → --- main 即将结束 --- → 销毁圆 → 销毁圆
//  （后创建的先销毁）
```

第 6 步：封装——getter / setter

封装 = 数据私有 + 通过受控方法访问。核心收益是：**可以在方法里做检查，拦住非法数据**。

```
class Student {
private:
    double score;
public:
    void setScore(double s) {
        if (s >= 0 && s <= 100) {
            score = s;                  // 合法才更新
        } else {
            cout << "成绩必须在 0~100 之间！" << endl;
        }
    }
    double getScore() {
        return score;                   // 只读返回
    }
};
Student s("小明", 18, 88);
s.setScore(120);      // 被拦住：成绩必须在 0~100 之间！
s.setScore(92);       // 合法，更新成功
cout << s.getScore(); // 92
```

【动画演示】封装：非法数据被 setter 拦截（请在学习软件中观看动画）

命名约定：getXxx() 取值，setXxx() 设值，一目了然。

封装的三大好处：

1. **防呆**：非法数据进不来（成绩 120、余额负数）
2. **灵活**：以后想加规则（比如改成绩要记录日志），只改 setter 一处
3. **稳定**：内部实现随便改，对外接口不变，调用方不用动

马上动手（练习 3）：给 BankAccount 加 getBalance() 和 withdraw()，取款时检查余额是否足够。

点开看参考答案

```
#include <iostream>
using namespace std;


class BankAccount {
private:
    string owner;
    double balance;
public:
    BankAccount(string o, double b) : owner(o), balance(b) {}


    void deposit(double m) {
        if (m > 0) balance += m;
    }
    bool withdraw(double m) {
        if (m > 0 && m <= balance) {   // 余额足够才放行
            balance -= m;
            return true;
        }
        cout << "余额不足或金额无效" << endl;
        return false;
    }
    double getBalance() { return balance; }
};


int main() {
    BankAccount acc("小明", 1000);
    acc.deposit(200);
    cout << "余额: " << acc.getBalance() << endl;   // 1200
    acc.withdraw(5000);                              // 被拦
    cout << "余额: " << acc.getBalance() << endl;   // 还是 1200
    return 0;
}
```

第 7 步：this 指针 + 静态成员

7.1 this 指针

`this` 指向"当前这个对象"。当参数名和成员变量同名时，用它区分：

```
class BankAccount {
private:
    double balance;
public:
    void deposit(double balance) {      // 参数和成员同名了
        this->balance += balance;       // this->balance 是成员，右边是参数
    }
};
```

- this->balance：当前对象的成员变量
- balance：参数

this 也可以用来"返回自己"，实现链式调用：

```
class Counter {
private:
    int value = 0;
public:
    Counter &add(int n) { value += n; return *this; }   // 返回自身引用
    int get() { return value; }
};


Counter c;
c.add(1).add(2).add(3);   // 链式调用
cout << c.get();          // 6
```

7.2 静态成员：属于"类"，不属于"对象"

普通成员每个对象各有一份；**静态成员（static）所有对象共享一份**：

```
class BankAccount {
private:
    static int totalAccounts;    // 所有账户共享这个计数器
    string owner;
public:
    BankAccount(string o) {
        owner = o;
        totalAccounts++;          // 每创建一个账户 +1
    }
    static int getTotal() {       // 静态成员函数
        return totalAccounts;
    }
};


// 静态成员必须在类外定义并初始化（重要！）
int BankAccount::totalAccounts = 0;
```

```
BankAccount a("小明"), b("小红"), c("小刚");
cout << BankAccount::getTotal();   // 3（用"类名::"调用）
```

理解：静态成员像是挂在"类"这个大门上的公告板，所有对象都能看，谁都能改；访问用 类名::，不需要对象。

马上动手（练习 4）：给 Circle 类加静态成员 count，统计一共创建了多少个圆。

点开看参考答案

```
#include <iostream>
using namespace std;


class Circle {
private:
    double radius;
    static int count;
public:
    Circle(double r) : radius(r) { count++; }
    static int getCount() { return count; }
};
int Circle::count = 0;


int main() {
    Circle c1(1), c2(2), c3(3);
    cout << "创建了 " << Circle::getCount() << " 个圆" << endl;  // 3
    return 0;
}
```

第 8 步：对象数组 + 综合项目：银行账户系统

8.1 对象数组

```
Student classRoom[3] = {
    Student("张三", 20, 78),
    Student("李四", 21, 85),
    Student("王五", 22, 90)
};


for (int i = 0; i < 3; i++) {
    classRoom[i].printInfo();     // 逐个调用成员函数
}
```

8.2 综合项目：迷你银行账户系统

把今天学的全用上，做一个"开两个账户、存款取款、显示总账户数"的小系统：

```
#include <iostream>
#include <string>
using namespace std;


class BankAccount {
private:
    string owner;
    double balance;
    static int totalAccounts;


public:
    BankAccount(string o, double b) : owner(o), balance(b) {
        totalAccounts++;
        cout << "开户成功: " << owner << endl;
    }


    ~BankAccount() {
        cout << "账户已注销: " << owner << endl;
    }


    void deposit(double money) {
        if (money > 0) {
            balance += money;
            cout << owner << " 存入 " << money << " 元" << endl;
        }
    }


    bool withdraw(double money) {
        if (money > 0 && money <= balance) {
            balance -= money;
            cout << owner << " 取出 " << money << " 元" << endl;
            return true;
        }
        cout << owner << " 取款失败: 余额不足" << endl;
        return false;
    }


    void show() {
        cout << owner << " 的余额: " << balance << " 元" << endl;
    }


    static int getTotal() { return totalAccounts; }
};


int BankAccount::totalAccounts = 0;


int main() {
```

```
    BankAccount acc1("小明", 1000);
    BankAccount acc2("小红", 500);


    acc1.deposit(200);
    acc1.withdraw(150);
    acc1.show();


    cout << "当前共有账户：" << BankAccount::getTotal() << " 个" << endl;
    return 0;
}
```

逐段精讲：

- private 数据 + public 方法：封装
- 构造函数里 totalAccounts++：静态成员统计开户数
- withdraw 里检查余额：防呆
- 析构函数输出提示：观察对象生命周期
- BankAccount::getTotal()：用类名访问静态方法

自己改一改（挑战）：

1. 加一个 transferTo(账户, 金额) 方法，实现转账（从自己账户扣款，给对方加款）
2. 加静态成员 totalMoney，统计所有账户总金额，每次存取款都更新

分级练习

练习 A（基础）：Date 日期类

设计 Date 类：年月日三个私有成员，构造函数初始化，方法 print() 输出"2026 年 8 月 16 日"格式。

点开看答案

```
#include <iostream>
using namespace std;


class Date {
private:
    int year, month, day;
public:
    Date(int y, int m, int d) : year(y), month(m), day(d) {}
    void print() {
        cout << year << "年" << month << "月" << day << "日" << endl;
    }
};
```

```
int main() {
    Date d(2026, 8, 16);
    d.print();
    return 0;
}
```

练习 B（进阶）：带检查的成绩类

设计 Score 类：私有 value；setValue 只在 0~100 时更新；getValue 返回；再提供 isPass()（≥60 为真）。

点开看答案

```
#include <iostream>
using namespace std;

class Score {
private:
    int value = 0;
public:
    void setValue(int v) {
        if (v >= 0 && v <= 100) value = v;
        else cout << "成绩无效" << endl;
    }
    int getValue() { return value; }
    bool isPass() { return value >= 60; }
};

int main() {
    Score s;
    s.setValue(120);    // 被拦
    s.setValue(85);
    cout << s.getValue() << " " << (s.isPass() ? "及格" : "不及格") << endl;
    return 0;
}
```

练习 C（挑战）：学生管理系统雏形

设计 Student 类 + 一个 StudentManager 类：管理器内部用数组（或第 6 天学了 vector 后换成 vector）存学生，提供 add 和 showAll。今天先尝试把管理器做出来（可以用固定大小数组，如最多 100 人）。

点开看答案

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
```

```
    string name;
    double score;
    Student() {}
    Student(string n, double s) : name(n), score(s) {}
};


class StudentManager {
private:
    Student list[100];      // 最多 100 人（学完 vector 后可升级）
    int count = 0;
public:
    void add(Student s) {
        if (count < 100) list[count++] = s;
    }
    void showAll() {
        for (int i = 0; i < count; i++) {
            cout << list[i].name << " " << list[i].score << endl;
        }
    }
};


int main() {
    StudentManager m;
    m.add(Student("小明", 88));
    m.add(Student("小红", 95));
    m.showAll();
    return 0;
}
```

自测（10 题）

1. 类和对象的区别？
2. public 和 private 的区别？为什么成员变量一般设 private？
3. 构造函数什么时候执行？它的名字有什么规定？
4. 析构函数什么时候执行？典型用途？
5. 什么是初始化列表？它写在哪？
6. this 是什么？什么时候用？
7. 静态成员和普通成员的区别？怎么访问静态成员？
8. 封装的好处有哪些？（至少 2 点）
9. 如果写了带参数的构造函数，还能不能 Student s; 这样创建对象？
10. 对象数组和普通数组的区别？

点开看答案

1. 类是模具/模板，对象是用模具造出来的具体实例。
2. public 外部可访问，private 外部不可访问；设 private 能保护数据不被乱改。
3. 创建对象时自动执行；名字与类名相同，无返回值类型。
4. 对象销毁时自动执行；典型用途是释放 new 的内存。
5. 构造函数签名后冒号跟着 `: 成员(值), ...`；用来直接初始化成员。
6. 指向当前对象的指针；参数与成员同名时用 this-> 区分。
7. 静态成员属于类、所有对象共享一份；用 `类名::成员` 访问。
8. 防呆（拦截非法数据）、内部实现可灵活修改、对外接口稳定。
9. 不能，一旦自定义了构造函数，默认无参构造函数就不存在了。
10. 对象数组的元素是对象，创建时逐个调用构造函数，可用下标访问并调用成员函数。

今日坑点汇总

坑	说明	正确做法
忘了初始化列表/构造函数	成员是垃圾值	用构造函数初始化
成员全 public	数据可以被乱改	数据 private + getter/setter
静态成员忘在类外定义	链接报错	`int 类名::成员 = 初始值;`
自定义构造后无参创建	编译错误	要么补默认构造，要么传参创建
析构忘释放内存	内存泄漏	在析构里 delete/delete[]
参数与成员同名	赋值给参数自己	用 this-> 区分

今日总结 + 明天预告

今天你进入了"面向对象"的世界：把数据和行为打包成类，用封装保护数据，用构造函数/析构函数管理生命周期，还用静态成员统计对象。**以后写程序，第一反应应该是"这里应该设计一个什么类"。**

明天（第 6 天）学什么？ 类的两大进阶能力——**继承**（子类复用父类）和**多态**（同一句话不同表现），再加上 STL 标准库的 vector、sort、map，写程序的效率会起飞。

睡前打卡： 不看答案，自己敲一遍"银行账户"综合项目，并加上第 8 步的"转账"挑战。

第 6 天：继承、多态 + STL 标准库

预计用时：5～6 小时 | 今天学完，你就掌握了面向对象的**两大王牌**（继承、多态）

和 C++ 最常用的**现成工具**（vector、sort、map）。从今天起，写程序会轻松很多。

今天学完，你能做到（打卡清单）

- [ ] 用继承复用代码（子类继承父类，理解构造顺序）
- [ ] 用虚函数实现多态（同一句代码，不同对象不同表现）
- [ ] 理解纯虚函数和抽象类
- [ ] 会用 vector（动态数组）代替普通数组
- [ ] 会用 sort 排序、map 做键值对

今天的路线图

```
第 1 步 回顾 + 为什么要继承（先看"重复代码"的痛点）
  ↓
第 2 步 继承语法 + protected + 构造顺序
  ↓
第 3 步 方法重写（override）
  ↓
第 4 步 多态：虚函数（virtual）——今天的重头戏
  ↓
第 5 步 纯虚函数与抽象类
  ↓
第 6 步 运算符重载（让自定义类型支持 +、<<）
  ↓
第 7 步 STL 三件套：vector / sort / map
```

第 1 步：为什么要继承？

假设你要设计一个动物园程序，有狗、猫、鸟。它们都有名字、都会"吃东西"，但"叫"的方式不同。不用继承的写法：

```
class Dog {
public:
    string name;
    void eat() { cout << name << " 在吃东西" << endl; }
    void bark() { cout << name << ": 汪汪! " << endl; }
};


class Cat {
public:
    string name;
    void eat() { cout << name << " 在吃东西" << endl; }   // ❌ 和 Dog 重复了
    void meow() { cout << name << ": 喵喵! " << endl; }
};
```

痛点：name 和 eat() 每个类都写一遍。以后加"游泳"功能，三个类都要改。

继承的思路：先写一个通用的 Animal（动物）类，狗、猫、鸟**继承**它，只写自己特有的部分：

```
Animal（父类/基类）：name + eat()
  ↑     ↑     ↑
  Dog      Cat      Bird
  bark()   meow()   fly()
```

术语： 被继承的叫**父类（基类）**，继承的叫**子类（派生类）**。

第 2 步：继承语法 + protected + 构造顺序

2.1 基本语法

```
class Animal {                  // 父类
public:
    string name;
    Animal(string n) : name(n) {}
    void eat() { cout << name << " 在吃东西" << endl; }
};


class Dog : public Animal {     // 子类继承父类
public:
    Dog(string n) : Animal(n) {}   // 先初始化父类部分
    void bark() { cout << name << ": 汪汪! " << endl; }
};
Dog d("旺财");
```

```
d.eat();      // 继承来的方法（父类的能力）
d.bark();     // 自己的方法
```

注意子类构造函数：Dog(string n) : Animal(n) {} —— **必须先调用父类的构造函数**，把父类的部分初始化好。

2.2 protected：给子类开的"后门"

权限	外部	子类	场景
public			对外接口
private			只有自己能用
protected			希望子类能用、但外部不能用

```
class Animal {
protected:
    string name;         // 子类能访问，外部不行
};
class Dog : public Animal {
public:
    void show() { cout << name; }    // ✅ 子类能访问 protected
};
int main() {
    Dog d("旺财");
    // cout << d.name;    // ❌ 外部不能访问 protected
}
```

2.3 构造顺序（重点）

创建子类对象时，先构造父类，再构造子类；销毁时相反（先子类后父类）。

```
class Animal {
public:
    Animal() { cout << "构造 Animal" << endl; }
    ~Animal() { cout << "析构 Animal" << endl; }
};
class Dog : public Animal {
public:
    Dog() { cout << "构造 Dog" << endl; }
    ~Dog() { cout << "析构 Dog" << endl; }
};
int main() {
    Dog d;
    return 0;
}
// 输出顺序：
```

```
// 构造 Animal → 构造 Dog → 析构 Dog → 析构 Animal
```

马上动手（练习 1）：定义 Vehicle（车：品牌 brand，启动方法 start）和子类 Car（额外有后备箱容量 trunkSize），演示继承和构造顺序。

点开看参考答案

```
#include <iostream>
#include <string>
using namespace std;


class Vehicle {
protected:
    string brand;
public:
    Vehicle(string b) : brand(b) { cout << "构造 Vehicle" << endl; }
    void start() { cout << brand << " 启动了" << endl; }
};


class Car : public Vehicle {
private:
    int trunkSize;
public:
    Car(string b, int t) : Vehicle(b), trunkSize(t) {
        cout << "构造 Car" << endl;
    }
    void showTrunk() { cout << brand << " 后备箱 " << trunkSize << "L" << endl; }
};


int main() {
    Car c("Tesla", 500);
    c.start();
    c.showTrunk();
    return 0;
}
```

第 3 步：方法重写（override）

子类可以**重写**父类的方法，改成自己的版本：

```
class Animal {
public:
    void speak() { cout << "动物发出声音" << endl; }
};


class Dog : public Animal {
public:
```

```
    void speak() { cout << "汪汪汪！" << endl; }   // 重写
};
Dog d("旺财");
d.speak();      // 输出"汪汪汪！"（调用的是 Dog 自己的版本）
```

但这里有个大坑——如果通过**父类指针**调用呢？

```
Animal *a = new Dog("旺财");
a->speak();     // ❌ 输出"动物发出声音"！（因为 speak 不是虚函数）
```

这就要引出今天的重头戏：**多态**。

第 4 步：多态——虚函数（virtual）

4.1 问题场景

假设有一个"动物列表"，想让每只动物都"叫一声"。用父类指针数组来统一管理：

```
Animal *animals[2];
animals[0] = new Dog("小黑");
animals[1] = new Cat("小橘");
```

我们希望能统一写 animals[i]->speak()，让狗叫狗声、猫叫猫声。**不写 virtual 时做不到**（调用的都是 Animal 版本）。

> 大白话：多态 = 同一个"播放"按钮，MP3 放音乐、电视放画面、手机放视频——**同一个动作，不同设备表现不同**。C++ 里就是同一个 speak()，狗叫狗声、猫叫猫声。

4.2 virtual 解决

在父类方法前加 virtual，子类重写时加 override：

```
class Animal {
public:
    virtual void speak() {           // virtual：允许子类重写并实现多态
        cout << "动物发出声音" << endl;
    }
};


class Dog : public Animal {
public:
    void speak() override {          // override：明确表示"我重写了父类方法"
        cout << "汪汪汪！" << endl;
    }
};
```

```
class Cat : public Animal {
public:
    void speak() override {
        cout << "喵喵喵～" << endl;
    }
};
Animal *a1 = new Dog("小黑");
Animal *a2 = new Cat("小橘");
a1->speak();   // 汪汪汪！（多态：按"真实类型"调用）
a2->speak();   // 喵喵喵～
delete a1;
delete a2;
```

多态的精髓一句话：用父类指针调用方法，执行的是"对象真实类型"的版本。

- 不加 virtual：编译时按"指针类型"（Animal）决定 → 调 Animal 版
- 加了 virtual：运行时按"对象实际类型"（Dog/Cat）决定 → 调对应版本

【动画演示】多态：同一个 speak()，狗叫狗声、猫叫猫声（请在学习软件中观看动画）

4.3 虚析构函数（重要！）

如果父类析构函数不是虚的，通过父类指针 delete 子类对象时，子类的析构函数不会执行！

```
class Animal {
public:
    virtual ~Animal() {}   // ✅ 父类析构加 virtual
};

Animal *a = new Dog("小黑");
delete a;    // 不写 virtual：只调 Animal 析构 → Dog 里 new 的内存泄漏！
             // 写了 virtual：先调 Dog 析构，再调 Animal 析构 ✅
```

规则：只要类里有虚函数，析构函数就加 virtual。这是 C++ 最经典的内存安全规范之一。

必做调试：在第 4.2 的代码里，给 a1->speak() 打断点，Step Into 进入，观察它进入的是 Dog 的 speak 而不是 Animal 的——**亲眼见证多态。**

马上动手（练习 2）：设计 Shape（形状，虚函数 area）→ Rectangle（宽高）和 Circle（半径），用父类指针数组分别计算面积。

点开看参考答案

```
#include <iostream>
using namespace std;

class Shape {
```

```
public:
    virtual double area() { return 0; }
    virtual ~Shape() {}
};


class Rectangle : public Shape {
private:
    double w, h;
public:
    Rectangle(double w_, double h_) : w(w_), h(h_) {}
    double area() override { return w * h; }
};


class Circle : public Shape {
private:
    double r;
public:
    Circle(double r_) : r(r_) {}
    double area() override { return 3.14159 * r * r; }
};


int main() {
    Shape *shapes[2];
    shapes[0] = new Rectangle(3, 4);
    shapes[1] = new Circle(5);


    for (int i = 0; i < 2; i++) {
        cout << "面积: " << shapes[i]->area() << endl;
        delete shapes[i];
    }
    return 0;
}
```

第 5 步：纯虚函数与抽象类

5.1 问题

Animal::speak() 里输出"动物发出声音"其实很尴尬——现实中"动物"是一个抽象概念，没有哪种动物真的这么叫。更合理的做法是：**父类只规定"必须有 speak()"，但不实现，强制子类实现。**

5.2 纯虚函数

在函数声明后面加 = 0：

```
class Animal {
public:
    virtual void speak() = 0;    // 纯虚函数：只声明，不实现
    virtual ~Animal() {}
```

```
};
```

后果：

1. Animal 变成**抽象类**，**不能创建对象**：Animal a; → 编译错误
2. **子类必须实现 `speak()`**，否则子类也是抽象类

```
class Dog : public Animal {
public:
    void speak() override { cout << "汪汪汪" << endl; }   // 必须实现
};
```

5.3 为什么有用?

它定义了一个**"标准接口"**：任何 Animal 的子孙，都保证能 speak()。写通用代码时就不用关心具体是谁：

```
void makeAllSpeak(Animal *list[], int n) {
    for (int i = 0; i < n; i++) {
        list[i]->speak();     // 不管具体是狗是猫，一定能 speak
    }
}
```

马上动手（练习 3）：把第 4.2 的 Animal 改成抽象类（speak 变纯虚），确认 Animal a; 会编译报错，然后正常使用 Dog/Cat。

点开看答案

```
#include <iostream>
using namespace std;


class Animal {
public:
    virtual void speak() = 0;    // 纯虚函数
    virtual ~Animal() {}
};


class Dog : public Animal {
public:
    void speak() override { cout << "汪汪汪" << endl; }
};
class Cat : public Animal {
public:
    void speak() override { cout << "喵喵喵" << endl; }
};


int main() {
    // Animal a;              // ❌ 抽象类不能实例化
    Animal *list[2] = { new Dog(), new Cat() };
    for (int i = 0; i < 2; i++) {
        list[i]->speak();
        delete list[i];
```

```
    }
    return 0;
}
```

第 6 步：运算符重载——让自定义类型支持 +、<<

问题：定义一个分数类型后，f1 + f2 能直接用吗？

```
struct Fraction {
    int top;      // 分子
    int bottom;   // 分母
};
Fraction f1{1, 2}, f2{1, 3};
// Fraction f3 = f1 + f2;   // ❌ 编译错误：不会加法
```

运算符重载就是教 C++ "这个类型怎么用 +"：

```
struct Fraction {
    int top;
    int bottom;
    Fraction(int t, int b) : top(t), bottom(b) {}
};


// 重载 +
Fraction operator+(const Fraction &a, const Fraction &b) {
    return Fraction(
        a.top * b.bottom + b.top * a.bottom,   // 通分后的分子
        a.bottom * b.bottom                    // 通分后的分母
    );
}


// 重载 <<  （让 cout 能直接输出分数）
ostream &operator<<(ostream &out, const Fraction &f) {
    out << f.top << "/" << f.bottom;
    return out;
}
Fraction f1(1, 2), f2(1, 3);
Fraction f3 = f1 + f2;
cout << f1 << " + " << f2 << " = " << f3 << endl;
// 输出：1/2 + 1/3 = 5/6
```

要点：

- operator+ 的写法：返回类型 operator 符号(参数)
- 参数用 const & 避免拷贝、保证不改原对象
- operator<< 第一个参数必须是 ostream &，返回它才能连续输出

运算符重载是"锦上添花"的能力，今天先会用、能看懂即可，不用深挖。

第 7 步：STL 三件套——vector / sort / map

STL（Standard Template Library）是 C++ 自带的"工具库"，里面最常用的三样，能让你的代码量减半。

> 大白话：vector 像**会自己变长的抽屉柜**——放满了就自动加一层，你永远不用操心"会不会挤不下"。

7.1 vector：会自动扩容的"智能数组"

为什么需要它？ 普通数组大小固定、可能越界、不能随便增删。vector 全部搞定。

```
#include <vector>


vector<int> nums;             // 一个空的"整数动态数组"
nums.push_back(30);           // 尾部添加
nums.push_back(10);
nums.push_back(20);


nums.size();                  // 元素个数：3
nums[0];                      // 按下标访问：30（和数组一样）
nums[0] = 99;                 // 修改
nums.erase(nums.begin());     // 删除第一个元素
```

遍历（两种方式）：

```
// 方式 1：传统下标
for (size_t i = 0; i < nums.size(); i++) {
    cout << nums[i] << " ";
}


// 方式 2：范围 for（C++11 起，最推荐）
for (int v : nums) {
    cout << v << " ";
}
```

range-based for 是神器：for (类型 变量 : 容器) 自动从头到尾遍历，不用管下标。

【动画演示】vector：push_back 逐个加入尾部（请在学习软件中观看动画）

7.2 sort：一行排序

```
#include <algorithm>


vector<int> nums = {30, 10, 20, 50, 40};
```

```
sort(nums.begin(), nums.end());          // 从小到大
sort(nums.rbegin(), nums.rend());        // 从大到小
```

begin() 是第一个元素，end() 是"最后一个的后面"（迭代器概念，先记住用法）。

7.3 map：键值对"字典"

```
#include <map>


map<string, int> scores;
scores["小明"] = 90;       // 键：小明，值：90
scores["小红"] = 95;
scores["小明"];            // 取值：90


// 遍历（kv.first 是键，kv.second 是值）
for (const auto &kv : scores) {
    cout << kv.first << "=" << kv.second << " ";
}
```

`auto`：让编译器自动推断类型，写起来省事。const auto &kv = "kv 是自动推断类型的只读引用"。

【动画演示】map：键值对存储，按键查值（请在学习软件中观看动画）

7.4 综合小例子

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;


int main() {
    vector<int> nums = {30, 10, 20, 50, 40};
    sort(nums.begin(), nums.end());       // 排序
    for (int v : nums) cout << v << " ";  // 10 20 30 40 50
    cout << endl;


    nums.push_back(25);                   // 动态加一个
    cout << "现在有 " << nums.size() << " 个数" << endl;
    return 0;
}
```

马上动手（练习 4）：输入若干个整数（以 -1 结束）存入 vector，去掉重复后从大到小输出。

提示：先 sort，再用 unique 配合 erase 去重（见答案）。

点开看参考答案

```
#include <iostream>
```

```cpp
#include <vector>
#include <algorithm>
using namespace std;


int main() {
    vector<int> nums;
    int x;
    while (cin >> x && x != -1) nums.push_back(x);


    sort(nums.begin(), nums.end());                                    // 先排序
    nums.erase(unique(nums.begin(), nums.end()), nums.end());          // 去重


    sort(nums.rbegin(), nums.rend());                                  // 降序
    for (int v : nums) cout << v << " ";
    cout << endl;
    return 0;
}
```

分级练习

练习 A（基础）：员工工资体系（多态）

设计基类 Employee（姓名、基础工资，虚函数 salary），派生 Manager（加奖金）和 Salesman（加提成），用父类指针数组输出每人工资。

点开看答案

```cpp
#include <iostream>
#include <vector>
using namespace std;


class Employee {
protected:
    string name;
    double base;
public:
    Employee(string n, double b) : name(n), base(b) {}
    virtual double salary() { return base; }
    virtual ~Employee() {}
};


class Manager : public Employee {
private:
    double bonus;
public:
    Manager(string n, double b, double bo) : Employee(n, b), bonus(bo) {}
    double salary() override { return base + bonus; }
```

```
};


class Salesman : public Employee {
private:
    double commission;
public:
    Salesman(string n, double b, double c) : Employee(n, b), commission(c) {}
    double salary() override { return base + commission; }
};


int main() {
    vector<Employee*> staff;
    staff.push_back(new Manager("张经理", 10000, 5000));
    staff.push_back(new Salesman("李销售", 3000, 8000));


    for (Employee *e : staff) {
        cout << e->salary() << endl;   // 15000 / 11000（多态！）
        delete e;
    }
    return 0;
}
```

练习 B（进阶）：vector 管理学生

定义一个 Student 结构体（姓名、成绩），用 vector 存 3 个学生，按成绩从高到低排序输出。

点开看答案

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;


struct Student {
    string name;
    double score;
};


int main() {
    vector<Student> stu = {
        {"小明", 88}, {"小红", 95}, {"小刚", 72}
    };


    sort(stu.begin(), stu.end(), [](const Student &a, const Student &b) {
        return a.score > b.score;      // 按成绩降序
    });


    for (const Student &s : stu) {
```

```
        cout << s.name << " " << s.score << endl;
    }
    return 0;
}
```

练习 C（挑战）：词频统计（map）

输入一句英文（getline 读整行），统计每个单词出现次数并输出。提示：用 stringstream 把句子按空格拆成单词。

点开看答案

```
#include <iostream>
#include <sstream>
#include <string>
#include <map>
using namespace std;


int main() {
    string line;
    cout << "输入英文句子：";
    getline(cin, line);


    map<string, int> freq;
    stringstream ss(line);        // 把字符串当"输入流"
    string word;
    while (ss >> word) {          // 按空格拆出一个个单词
        freq[word]++;
    }


    for (const auto &kv : freq) {
        cout << kv.first << " : " << kv.second << endl;
    }
    return 0;
}
```

自测（10 题）

1. 继承的语法是什么？子类构造函数为什么要先调用父类构造函数？
2. protected 和 private 的区别？
3. 创建子类对象时，构造和析构的顺序？
4. 虚函数的作用？不加 virtual 会怎样？
5. override 关键字有什么用？
6. 什么是抽象类？能创建对象吗？

7. 为什么父类析构函数要加 virtual?
8. vector 和普通数组比，最大的优势是什么?
9. for (int v : nums) 这种循环叫什么? 适合什么场景?
10. map 里的 kv.first 和 kv.second 分别是什么?

点开看答案

1. class 子类 : public 父类；要先初始化父类部分的成员。
2. private 子类也不能访问，protected 子类可以访问（外部都不行）。
3. 先父类后子类构造；销毁相反，先子类后父类。
4. 实现多态；不加 virtual，父类指针调用的是父类版本。
5. 明确声明"重写父类方法"，编译器帮忙检查写没写错。
6. 含纯虚函数的类；不能创建对象，只能被继承。
7. 否则通过父类指针 delete 子类对象时，子类析构不会执行，可能内存泄漏。
8. 自动扩容、自动管理内存、配合 sort 等算法。
9. 范围 for（range-based for）；遍历容器的每个元素。
10. 键和值。

今日坑点汇总

坑	说明	正确做法
忘写 virtual	多态失效，调父类版本	父类方法加 virtual
父类析构不加 virtual	delete 时子类析构不执行	有虚函数就加虚析构
抽象类实例化	Animal a; 编译错误	抽象类只能被继承
子类忘实现纯虚函数	子类也是抽象类	子类必须实现
vector 越界	nums[5] 越界访问	用 at() 或先判断 size
sort 忘 #include <algorithm>	编译报错	用 sort 记得包含头文件

今日总结 + 明天预告

今天你拿到了三张王牌：**继承**复用代码、**多态**统一处理不同类型、**STL** 让写代码又快又稳。到这里，C++ 的核心语法你已经全部学完了。

明天（第 7 天）学什么？ 最后一天综合实战：**文件读写 + 学生成绩管理系统**，把 7 天所学全部串成一个大项目，做完它，这 7 天就圆满了。

睡前打卡： 用 vector + sort 重写"第 5 天的学生管理器"（把固定数组换成 vector），体会工具带来的轻松感。

第 7 天：综合项目实战 + 查漏补缺

预计用时：6～8 小时（最后一天，全力以赴） | 今天把 7 天所学**全部串起来**：

文件读写 + 一个完整的学生成绩管理系统。做完它，你就真正入门 C++ 了。

今天学完，你能做到（打卡清单）

- [ ] 会用 fstream 读写文件（数据能保存、能读回）
- [ ] 能独立设计"菜单式"管理程序
- [ ] 完成学生成绩管理系统（增删改查 + 排序 + 文件保存）
- [ ] 对照自查清单找出自己的薄弱点并补上
- [ ] 制定出之后的进阶学习路线

今天的路线图

```
第 1 步 回顾 7 天 + 今天要做什么
  ↓
第 2 步 文件读写：ofstream 写入 / ifstream 读取
  ↓
第 3 步 项目设计：需求分析 + 模块划分
  ↓
第 4 步 搭建骨架：结构体 + vector + 主菜单
  ↓
第 5 步 逐个实现功能：添加 → 显示 → 查找 → 修改 → 删除
  ↓
第 6 步 排序 + 文件保存/读取
  ↓
第 7 步 查漏补缺：7 天自查清单
  ↓
第 8 步 验收标准 + 后续进阶路线
```

第 1 步：回顾 + 今天要做什么

7 天你学了什么？一条线串起来：

```
第 1 天 变量/输入输出/运算符   →  让程序"会说话、会算数"
第 2 天 分支/循环          →  让程序"会判断、会重复"
第 3 天 函数/数组/字符串     →  让程序"有结构、能批量处理"
第 4 天 指针/引用/动态内存    →  让程序"能管理内存"
第 5 天 类与对象          →  让程序"会封装世界"
第 6 天 继承/多态/STL          →  让程序"能复用、有工具"
第 7 天 文件读写 + 综合项目    →  让程序"能真正投入使用"
```

今天做的大项目会用到所有这些知识，做的时候对照一下：哪一步卡住了，就说明哪天的知识没吃透——这正好帮你查漏补缺。

第 2 步：文件读写——让数据"活过"程序运行

问题：程序一关，内存里的数据全没了。学生成绩一关软件就丢，显然不合格。**文件读写**就是解决这个的。

2.1 写入文件（ofstream）

```
#include <fstream>      // 文件读写头文件


ofstream out("students.txt");    // 打开（或创建）文件，准备写入
if (!out) {                      // 打开失败检查（重要！）
    cout << "打开文件失败！" << endl;
    return 1;
}


out << "张三" << " " << 90 << endl;    // 像 cout 一样写入
out << "李四" << " " << 85 << endl;


out.close();                          // 关闭（养成好习惯）
```

要点：

- ofstream：output file stream，**输出**到文件
- 用法和 cout 几乎一样：out << 内容
- 文件不存在会自动创建；**存在会被覆盖清空**
- 用前判断 if (!out)，用后 close()

2.2 读取文件（ifstream）

```
ifstream in("students.txt");
if (!in) {
    cout << "文件不存在！" << endl;
    return 1;
}


string name;
int score;
while (in >> name >> score) {      // 像 cin 一样读，读不到就退出循环
    cout << name << " " << score << endl;
}
in.close();
```

要点：

- ifstream：input file stream，从文件**读取**
- 用法和 cin 几乎一样：in >> 变量
- while (in >> ...)：读到文件末尾自动停止，这是最常用的读法

【动画演示】文件读写：程序 ⇄ 磁盘文件（请在学习软件中观看动画）

2.3 追加模式

不想覆盖原文件、只想在后面加内容时，用 ios::app：

```
ofstream out("log.txt", ios::app);   // append：追加模式
out << "新增一条记录" << endl;
out.close();
```

2.4 文件存哪了？

文件保存在"程序的工作目录"。在 Xcode 里运行，通常是项目的 Debug 文件夹。找它的办法：程序里输出当前路径，或者直接在"访达"里搜索文件名 students.txt。

马上动手（练习 1）：写一个程序：第一次运行输入 3 个名字写入文件；第二次运行从文件读出来显示。

点开看参考答案

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;


int main() {
    // 写
    ofstream out("names.txt");
    for (int i = 0; i < 3; i++) {
        string n;
        cin >> n;
```

```
        out << n << endl;
    }
    out.close();


    // 读
    ifstream in("names.txt");
    string n;
    while (in >> n) {
        cout << "读回: " << n << endl;
    }
    in.close();
    return 0;
}
```

第 3 步：项目设计——需求分析与模块划分

项目：学生成绩管理系统

功能需求：

1. 添加学生（学号、姓名、语文/数学/英语成绩）
2. 显示所有学生（表格，含总分）
3. 按学号查找
4. 修改成绩
5. 删除学生
6. 按总分排序
7. 保存到文件 / 启动时读回
8. 退出

先想清楚数据结构：

```
struct Student {
    string id;          // 学号（唯一，当"主键"用）
    string name;        // 姓名
    double chinese;     // 语文
    double math;        // 数学
    double english;     // 英语
    double total() const { return chinese + math + english; }  // 计算总分
};
```

再想清楚用什么容器： 第 6 天学的 vector<Student>——能动态增删，完美匹配。

模块划分（每个功能一个函数）：

```
main            : 主菜单循环，调用各功能
loadFromFile    : 启动时读文件
saveToFile      : 退出时写文件
addStudent      : 添加
showAll         : 显示
findIndexById   : 按学号找位置（辅助函数）
findStudent     : 查找并显示
modifyStudent   : 修改
deleteStudent   : 删除
sortByTotal     : 排序
```

编程思维：先想清楚"数据长什么样、有哪些操作"，再动手写代码。这比上来就敲重要得多。

第 4 步：搭建骨架——结构体 + vector + 主菜单

先搭一个能跑起来的框架（骨架阶段不写具体功能，先让菜单转起来）：

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>
#include <iomanip>
using namespace std;


struct Student {
    string id;
    string name;
    double chinese, math, english;
    double total() const { return chinese + math + english; }
};


vector<Student> students;          // 全局数据（项目阶段为了简单，允许全局）
const string FILE_NAME = "students.txt";


void showMenu() {
    cout << "\n======== 学生成绩管理系统 ========" << endl;
    cout << "  1. 添加学生" << endl;
    cout << "  2. 显示所有学生" << endl;
    cout << "  3. 按学号查找" << endl;
    cout << "  4. 修改成绩" << endl;
    cout << "  5. 删除学生" << endl;
```

```
    cout << "  6. 按总分排序" << endl;
    cout << "  0. 保存并退出" << endl;
    cout << "================================" << endl;
    cout << "请选择: ";
}


int main() {
    int choice = -1;
    while (choice != 0) {
        showMenu();
        cin >> choice;
        switch (choice) {
            case 1: /* addStudent(); */ break;
            case 2: /* showAll(); */ break;
            // ... 其它选项
            case 0: cout << "再见! " << endl; break;
            default: cout << "无效选项" << endl; break;
        }
    }
    return 0;
}
```

先编译运行：菜单能来回切换、能退出，骨架就 OK 了。接下来把每个 case 补成真函数。

第 5 步：逐个实现功能

5.1 添加学生

```
bool idExists(const string &id) {          // 学号是否已存在（避免重复）
    for (const Student &s : students) {
        if (s.id == id) return true;
    }
    return false;
}


void addStudent() {
    Student s;
    cout << "请输入学号: ";
    cin >> s.id;
    if (idExists(s.id)) {
        cout << "该学号已存在，添加失败! " << endl;
        return;
    }
    cout << "请输入姓名: ";
    cin >> s.name;
    cout << "请输入语文成绩: ";
```

```
    cin >> s.chinese;
    cout << "请输入数学成绩: ";
    cin >> s.math;
    cout << "请输入英语成绩: ";
    cin >> s.english;


    students.push_back(s);          // 第 6 天学的 vector!
    cout << "添加成功! " << endl;
}
```

用到第 2 天的分支、第 3 天的函数、第 6 天的 vector 和 range-for。

5.2 显示所有学生（表格）

```
void showAll() {
    if (students.empty()) {                    // vector 判空
        cout << "目前没有学生记录。" << endl;
        return;
    }
    cout << "------------------------------------------------" << endl;
    cout << left << setw(10) << "学号" << setw(10) << "姓名"
         << setw(8) << "语文" << setw(8) << "数学"
         << setw(8) << "英语" << setw(8) << "总分" << endl;
    cout << "------------------------------------------------" << endl;
    for (const Student &s : students) {
        cout << left << setw(10) << s.id << setw(10) << s.name
             << setw(8) << s.chinese << setw(8) << s.math
             << setw(8) << s.english << setw(8) << s.total() << endl;
    }
    cout << "------------------------------------------------" << endl;
}
```

新知识 `setw(10)`：让输出占 10 个字符宽度，配合 left 左对齐，表格才整齐。需要 #include <iomanip>。

5.3 按学号查找（+ 辅助函数）

```
int findIndexById(const string &id) {     // 找到返回下标，找不到返回 -1
    for (size_t i = 0; i < students.size(); i++) {
        if (students[i].id == id) return (int)i;
    }
    return -1;
}


void findStudent() {
    string id;
    cout << "请输入要查找的学号: ";
    cin >> id;
    int idx = findIndexById(id);
```

```
    if (idx == -1) {
        cout << "没有找到该学号的学生。" << endl;
        return;
    }
    Student &s = students[idx];
    cout << "姓名: " << s.name << ", 语文: " << s.chinese
         << ", 数学: " << s.math << ", 英语: " << s.english
         << ", 总分: " << s.total() << endl;
}
```

5.4 修改成绩

```
void modifyStudent() {
    string id;
    cout << "请输入要修改的学号: ";
    cin >> id;
    int idx = findIndexById(id);
    if (idx == -1) {
        cout << "没有找到该学号的学生。" << endl;
        return;
    }
    Student &s = students[idx];        // 引用: 直接改 vector 里的数据
    cout << "当前成绩: 语文 " << s.chinese << ", 数学 " << s.math
         << ", 英语 " << s.english << endl;
    cout << "请输入新的语文成绩: ";
    cin >> s.chinese;
    cout << "请输入新的数学成绩: ";
    cin >> s.math;
    cout << "请输入新的英语成绩: ";
    cin >> s.english;
    cout << "修改成功! " << endl;
}
```

注意 Student &s = students[idx]; 用了第 4 天的**引用**——这样改 s 就是改 vector 里的原数据。

5.5 删除学生

```
void deleteStudent() {
    string id;
    cout << "请输入要删除的学号: ";
    cin >> id;
    int idx = findIndexById(id);
    if (idx == -1) {
        cout << "没有找到该学号的学生。" << endl;
        return;
    }
    cout << "确定删除 " << students[idx].name << " 吗? (y/n): ";
    char ch;
    cin >> ch;
```

```
    if (ch == 'y' || ch == 'Y') {
        students.erase(students.begin() + idx);   // vector 删除
        cout << "已删除。" << endl;
    } else {
        cout << "已取消。" << endl;
    }
}
```

第 6 步：排序 + 文件保存/读取

6.1 按总分排序（用 sort + lambda）

```
void sortByTotal() {
    sort(students.begin(), students.end(),
         [](const Student &a, const Student &b) {
             return a.total() > b.total();     // 总分高的排前面
         });
    cout << "已按总分从高到低排序。" << endl;
    showAll();
}
```

lambda（匿名函数）快速理解：[](参数){ 函数体 } 就是"随手写一个用完即走的小函数"。这里的 sort 需要知道"怎么比较两个 Student"，我们通过 lambda 告诉它：按 total() 比。

6.2 保存与读取

```
void saveToFile() {
    ofstream out(FILE_NAME);
    if (!out) {
        cout << "保存失败：无法打开文件！" << endl;
        return;
    }
    out << students.size() << endl;                  // 第一行写人数
    for (const Student &s : students) {
        out << s.id << " " << s.name << " "
            << s.chinese << " " << s.math << " " << s.english << endl;
    }
    out.close();
    cout << "已保存 " << students.size() << " 条记录。" << endl;
}


void loadFromFile() {
    ifstream in(FILE_NAME);
    if (!in) {
        cout << "没有找到存档，从空名单开始。" << endl;
        return;
    }
    int count = 0;
```

```
    in >> count;
    students.clear();                                   // 先清空
    for (int i = 0; i < count; i++) {
        Student s;
        in >> s.id >> s.name >> s.chinese >> s.math >> s.english;
        students.push_back(s);
    }
    in.close();
    cout << "已载入 " << students.size() << " 条记录。" << endl;
}
```

保存格式设计：第一行人数，后面每行一个学生（字段用空格分隔）。这种"自己定义的简单文本格式"是最常见的入门做法。

6.3 拼装 main

```
int main() {
    loadFromFile();          // 启动时读回数据


    int choice = -1;
    while (choice != 0) {
        showMenu();
        cin >> choice;
        switch (choice) {
            case 1: addStudent();      break;
            case 2: showAll();         break;
            case 3: findStudent();     break;
            case 4: modifyStudent();   break;
            case 5: deleteStudent();   break;
            case 6: sortByTotal();     break;
            case 0: saveToFile();      break;   // 退出前保存
            default: cout << "无效选项，请重新输入。" << endl; break;
        }
    }
    cout << "感谢使用，再见！" << endl;
    return 0;
}
```

完整代码就在软件"代码实验室"的 `day07_final_project.cpp` 里，可以直接运行。

运行测试流程（一定要完整走一遍）：

1. 添加 3 个学生（其中一个学号故意重复，验证拦截）
2. 显示 → 看表格是否整齐
3. 查找一个存在的、一个不存在的
4. 修改成绩再显示
5. 删除一个再显示
6. 按总分排序

7. 退出 → 重新运行 → 数据还在吗？（应该在！）

第 7 步：查漏补缺——7 天自查清单

花 30 分钟，逐条问自己"能不能不看资料写出来"：

- [ ] cout / cin / 变量 / 5 种基本类型（第 1 天）
- [ ] if / switch / for / while / do-while / break / continue（第 2 天）
- [ ] 函数 / 值传递 / 递归 / 数组 / 冒泡排序 / string（第 3 天）
- [ ] 地址 / 指针 / 解引用 / 引用 / new-delete（第 4 天）
- [ ] 类 / 构造析构 / public-private / 封装 / this / 静态成员（第 5 天）
- [ ] 继承 / 虚函数 / 多态 / vector / sort / map（第 6 天）
- [ ] 文件读写 / 综合项目（第 7 天）

哪里打不了勾，回去重新敲那天的示例代码，再做两道那天的练习题。 宁可放慢，也要把基础补扎实。

再补两个高频面试概念（快速过一遍）：

- **值传递 vs 引用传递 vs 指针传递**：副本 / 别名 / 地址，改外面数据的三种选择
- **栈 vs 堆**：栈自动管理、快、大小有限；堆要 new/delete、灵活、要防泄漏

第 8 步：验收标准 + 后续进阶路线

7 天后你应该能做到

1. 30 分钟内，不查资料，从零写出"猜数字"游戏
2. 能讲清楚：指针是什么、值 vs 地址的区别
3. 能用类封装现实概念（学生、账户、圆）
4. 能用 vector + sort 处理一批数据
5. 综合项目能独立演示：增删改查 + 排序 + 文件保存
6. 遇到报错，能读懂错误信息、会用断点定位问题

如果你这 6 条都能做到，这一周就真的"学会 C++ 基础"了。 注意：7 天学会的是"基础 + 能上手写小程序"，离"熟练的 C++ 工程师"还有距离，但你已经具备了继续自学的一切能力。

接下来怎么进阶

阶段	内容	建议
第 2 周	刷题巩固	LeetCode 简单题每天 3 道（两数之和、回文数等）
第 3～4 周	深入 C++	智能指针 unique_ptr/shared_ptr、const 进阶、模板入门
第 5～8 周	图形界面	学 Qt（跨平台 C++ 界面库），做记事本/计算器 GUI
长期	专项方向	游戏（Unreal）、系统开发（操作系统/驱动）、算法竞赛

资源推荐：

- 经典教材：《C++ Primer》（第 5 版）——学完这套再看它，会顺畅很多
- 免费在线：cppreference.com（查函数用法神器）、cplusplus.com
- 练习平台：LeetCode、洛谷（中文算法题库）

今日总结 + 写在最后

这一周，你从"打开 Xcode"走到了"写一个能存文件的完整管理系统"。最后记住三条心法：

1. **写代码 > 看代码**：所有示例至少亲手敲一遍
2. **报错不可怕**：读懂报错 + 断点调试，比背语法更有用
3. **坚持动手**：每天固定时间敲代码，哪怕只写 20 分钟

恭喜完成 7 天 C++ 快速入门！去完成今天的打卡，然后打开"代码实验室"，把学生管理系统亲手敲一遍——它是你这 7 天最好的毕业作品

C++ 术语速查

教程里出现过的术语，这里用**一句话**讲清楚。

想不起来的时候，随时回来查。按拼音/字母顺序排列。

程序与运行

术语	一句话解释	出现在
程序	给电脑的一串指令，按顺序执行	预备课
编程	写这些指令的过程	预备课
编译	把代码"翻译"成电脑能执行的机器语言	预备课/第 1 天
运行	执行编译好的程序，得到结果	预备课/第 1 天
编译错误	代码写错（如少分号），编译阶段被电脑指出	第 1 天
运行时错误	能编译但运行中出错（如除数为 0）	第 1 天
调试	找出并修复 bug 的过程	第 1 天
断点	打在代码行上，运行到这里会暂停，方便查看	第 1 天

数据

术语	一句话解释	出现在
变量	带名字的"储物盒"，用来存数据，值可以变	第 1 天
常量	值不能变的量	第 1 天
数据类型	数据种类：整数、小数、字符、真假、字符串	第 1 天
表达式	由变量/常量/运算符组成的一段"算式"	第 1 天
语句	以分号结尾的一条指令	第 1 天
作用域	变量"活着"的范围（花括号内）	第 3 天
全局变量	定义在所有函数外的变量，到处能用	第 3 天
局部变量	定义在函数/花括号内的变量	第 3 天

运算与流程

术语	一句话解释	出现在
运算符	+ - * / % 等"运算符号"	第 1 天
取余 %	除法剩下的余数，如 7 % 3 = 1	第 1 天
分支	根据条件走不同的路（if/switch）	第 2 天
循环	反复执行同一段代码（for/while）	第 2 天
死循环	条件永远成立，程序停不下来	第 2 天

函数

术语	一句话解释	出现在
函数	打包好的一段代码，起个名字随时调用	第 3 天
参数	调用函数时传进去的数据	第 3 天
返回值	函数算完后送回的结果	第 3 天
值传递	传的是副本，函数里改了不影响外面	第 3 天
引用传递	传的是"别名"，函数里改的就是外面	第 4 天
递归	函数自己调用自己，把大问题拆小	第 3 天
重载	同名函数、参数不同，编译器自动选	第 3 天

数据容器

术语	一句话解释	出现在
数组	一排同类型的格子，按下标访问（从 0 开始）	第 3 天
下标	格子的编号，如 arr[0] 是第 1 个	第 3 天
越界	访问了不存在的格子（下标超范围）	第 3 天
字符串 string	一串文字	第 3 天
容器 vector	能自动变大变小的"智能数组"	第 6 天
键值对 map	"字典"：用键查值	第 6 天

内存

术语	一句话解释	出现在
地址	变量在内存里的"门牌号"	第 4 天
指针	存地址的变量，可以通过它修改原数	第 4 天

	据	
解引用 *	顺着地址取出/修改里面的值	第 4 天
引用 &	给变量起"别名"，直接操作原变量	第 4 天
空指针 nullptr	不指向任何地方的安全指针	第 4 天
野指针	没初始化就用的指针，指向随机地址（危险）	第 4 天
动态内存 new/delete	运行时按需申请/释放内存	第 4 天
内存泄漏	new 了不 delete，内存悄悄流失	第 4 天
悬空指针	内存已释放但指针还留着旧地址	第 4 天

面向对象

术语	一句话解释	出现在
类 class	"模具"：描述有什么数据、能做什么	第 5 天
对象	用模具造出来的具体"实例"	第 5 天
成员变量/函数	属于某个类的数据/函数	第 5 天
封装	数据私有 + 通过受控方法访问，保护数据	第 5 天
构造函数	创建对象时自动执行的初始化函数	第 5 天
析构函数	对象销毁时自动执行的清理函数	第 5 天
继承	子类复用父类的代码	第 6 天
多态	同一句代码，不同对象表现不同	第 6 天
虚函数	加了 virtual，让多态生效	第 6 天
抽象类	含纯虚函数的类，只能被继承不能实例化	第 6 天

其它

术语	一句话解释	出现在
头文件 #include	引入工具库，如 <iostream> 提供输入输出	第 1 天
命名空间	给名字加"姓"，避免重名冲突	第 1 天
注释 //	写给人类看的说明，电脑忽略	第 1 天
格式化输出 setw	让输出占固定宽度，表格更整齐	第 7 天
文件读写 fstream	把数据存到磁盘/从磁盘读回	第 7 天
lambda	随手写的"一次性小函数"，用于 sort 等	第 7 天

记不住很正常，**用的时候回来查**，查几次自然就记住了。